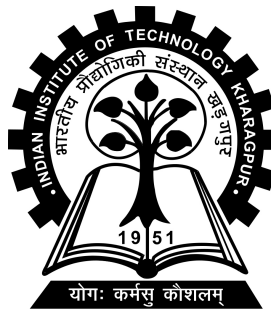


Inclusion Prediction and Classification in Steel Microstructures Using Deep Learning

Project-I (MT47001) report submitted to
Indian Institute of Technology Kharagpur
in partial fulfilment for the award of the degree of
Bachelor of Technology
in
Metallurgical and Materials Engineering
by

Name	Roll No.
Biswas Tuhin Manik	22MT3AI38
Kamal Nayan Bajoria	22MT10027
Anshuman Sahoo	22MT10008
Suryansh Singh	22MT10057

Under the supervision of
Professor Sumantra Mandal



Department of Metallurgical and Materials Engineering

Indian Institute of Technology Kharagpur

Autumn Semester, 2025-26

November 28, 2025

DECLARATION

I certify that

- (a) The work contained in this report has been done by us under the guidance of our supervisor.
- (b) The work has not been submitted to any other Institute for any degree or diploma.
- (c) We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.
- (d) Whenever we have used materials (data, theoretical analysis, figures, and text) from other sources, we have given due credit to them by citing them in the text of the thesis and giving their details in the references. Further, we have taken permission from the copyright owners of the sources, whenever necessary.

Date: November 28, 2025

Place: Kharagpur

Biswas Tuhin Manik
Kamal Nayan Bajoria
Anshuman Sahoo
Suryansh Singh

**DEPARTMENT OF METALLURGICAL AND
MATERIALS ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
KHARAGPUR - 721302, INDIA**



CERTIFICATE

This is to certify that the project report entitled “**Inclusion Prediction and Classification in Steel Microstructures Using Deep Learning**” submitted by:

Name	Roll No.
Biswas Tuhin Manik	22MT3AI38
Kamal Nayan Bajoria	22MT10027
Anshuman Sahoo	22MT10008
Suryansh Singh	22MT10057

to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Bachelor of Technology in Metallurgical and Materials Engineering is a record of bona fide work carried out by them under my supervision and guidance during Autumn Semester, 2025-26.

Date: November 28, 2025

Place: Kharagpur

Professor Sumantra Mandal
Department of Metallurgical and
Materials Engineering
Indian Institute of Technology Kharagpur
Kharagpur - 721302, India

Abstract

Names of students:

Roll Nos:

Biswas Tuhin Manik

22MT3AI38

Kamal Nayan Bajoria

22MT10027

Anshuman Sahoo

22MT10008

Suryansh Singh

22MT10057

Degree for which submitted: Bachelor of Technology

Department: Department of Metallurgical and Materials Engineering

Thesis title: Inclusion Prediction and Classification in Steel Microstructures
Using Deep Learning

Thesis supervisor: Professor Sumantra Mandal

Month and year of thesis submission: November 28, 2025

Non-metallic inclusions in steel significantly influence mechanical properties, machinability, fatigue life, and overall reliability of critical components.

This project develops a two-stage deep-learning pipeline for (1) pixel-wise inclusion segmentation and (2) morphology-based multi-label classification using metallographic images.

For segmentation, U-Net, U-Net++, Attention U-Net, ENet and Mask R-CNN were trained on a small but augmented Roboflow dataset, achieving best performance with Mask R-CNN (pixel accuracy 0.9630, Dice 0.7974).

For classification, a ResNet50-based multi-label classifier was trained on 107 acquired micrographs (expanded via augmentation to ~ 500 –600 images), reaching a test F1-score of 0.7018 with good per-class metrics for sulfide, alumina, silicate, and globular oxide inclusions.

The results demonstrate that even with limited data, carefully designed preprocessing, augmentation, and appropriate architectures can deliver promising performance for automated inclusion prediction and support metallurgical decision-making.

Keywords: non-metallic inclusions, steel, segmentation, Mask R-CNN, U-Net, ResNet50, multi-label classification, metallography, ASTM E45.

Acknowledgements

We sincerely thanks our project guide supervisor, Professor Sumantra Mandal, for his invaluable insights and constructive feedback which made this project a successful endeavour. We also thank Mr. Roshan Jha, Research Scholar, for his guidance throughout the project. Furthermore, we also thank the Department of Metallurgical and Materials Engineering for granting us an opportunity and resources for pursuing this project. We finally extend our gratitude to our family and friends for their support.

Contents

Declaration	i
Certificate	ii
Abstract	iii
Acknowledgements	v
Contents	v
List of Figures	x
List of Tables	xii
Abbreviations	xiii
Symbols	xiv
1 Theory of Inclusions and Metallographic Basis	1
1.1 Non-metallic inclusions in steel	1
1.2 Classification of Inclusions	2
1.2.1 Origins	2
1.2.2 Morphological Classification (ASTM E45)	3
1.2.2.1 Sulfide Inclusions (Type A)	3
1.2.2.2 Alumina Inclusions (Type B)	3
1.2.2.3 Silicate Inclusions (Type C)	4
1.2.2.4 Globular Oxide Inclusions (Type D)	4
1.3 Problems Caused by Inclusions	4
1.4 Methods of detecting and characterizing inclusions	5
1.5 ASTM Standards	6
1.6 Sample preparation and polishing in this project	7
2 Segmentation Models and Results	9
2.1 Objective	9

2.2	Dataset acquisition and augmentation for segmentation	9
2.3	COCO-format annotations and data pipeline	10
2.4	Overview of evaluated segmentation architectures	10
2.4.1	U-Net	10
2.4.2	U-Net++	11
2.4.3	Attention U-Net	11
2.4.4	ENet (Efficient Neural Network)	11
2.4.5	Mask R-CNN	11
2.5	Data loaders, training configuration, and metrics	12
2.6	Quantitative segmentation results	12
2.7	Analysis of Results	13
3	Detailed Analysis of Segmentation Models	14
3.1	Common Building Blocks Used in All CNN Models	14
3.1.1	Convolution	14
3.1.2	Batch Normalization	14
3.1.3	Activation Functions	15
	ReLU:	15
	PReLU:	15
3.1.4	Pooling	15
	Max pooling:	15
	Average pooling:	15
3.1.5	5. Convolution Transpose (Deconvolution)	15
3.1.6	Dropout	16
3.1.7	Sigmoid	16
3.1.8	Softmax	16
3.1.9	Binary Cross-Entropy Loss	16
3.2	U-Net	16
3.2.1	Architecture Components	16
3.2.2	How U-Net Works	18
3.2.3	Mathematical Formulation	19
3.3	U-Net++	21
3.3.1	Key Improvements	22
3.3.2	Mathematical Formulation	22
3.4	Attention U-Net	25
3.4.1	Mathematical Formulation	25
3.5	ENet (Efficient Neural Network)	29
3.5.1	Architectural Features	29
3.5.2	Mathematical Formulation	30
3.6	Mask R-CNN	35
3.6.1	How Mask R-CNN Works	35
3.6.2	Mathematical Formulation	36

4	Need for Classification and Inclusion Types	40
4.1	Motivation for going beyond segmentation	40
4.2	Inclusion types for classification	41
4.3	Multi-label nature of the problem	42
4.4	Sample Preparation and Data Collection	42
5	Inclusion Classification	43
5.1	Introduction	43
5.1.1	Motivation and Problem Statement	43
5.1.2	Deep Learning Opportunity	44
5.1.3	Multi-Label Challenge	45
5.1.4	Research Contributions	45
5.2	Theoretical Background	46
5.2.1	Machine Learning Fundamentals	46
5.2.1.1	Supervised Learning Paradigm	46
5.2.1.2	Classification vs Regression	47
5.2.1.3	Generalization and Overfitting	47
5.2.2	Artificial Neural Networks	48
5.2.2.1	Neuron Model and Activation Functions	48
5.2.2.2	Multi-Layer Perceptrons (MLPs)	49
5.2.2.3	Backpropagation Algorithm	49
5.2.3	Convolutional Neural Networks (CNNs)	50
5.2.3.1	Convolution Operation	50
5.2.3.2	Pooling Operations	51
5.2.3.3	Typical CNN Architecture	51
5.2.4	ResNet and Residual Learning	52
5.2.4.1	Vanishing Gradient Problem	52
5.2.4.2	Residual Connections (Skip Connections)	52
5.2.4.3	ResNet50 Architecture	53
5.2.5	Transfer Learning	54
5.2.5.1	Pre-Training and Fine-Tuning Paradigm	54
5.2.5.2	Learning Rate for Transfer Learning	55
5.2.5.3	Data Efficiency	55
5.2.6	Multi-Label Learning	56
5.2.6.1	Multi-Label Problem Formulation	56
5.2.6.2	Binary Cross-Entropy Loss (Multi-Label)	56
5.2.6.3	Inference: Threshold Selection	56
5.2.7	Optimization: Adam Optimizer	57
5.2.7.1	Gradient Descent Variants	57
5.2.7.2	Adam (Adaptive Moment Estimation)	58
5.2.7.3	Learning Rate Scheduling	59
5.2.8	Deep Learning Implementation	60

5.2.8.1	Model Architecture Design	60
5.2.8.2	Feature Extraction: ResNet50 Backbone	60
5.2.8.3	Classifier Head: Custom Multi-Label Predictor	61
5.3	Results and Empirical Analysis	64
5.3.1	Dataset Statistics	64
5.3.2	Training Dynamics	65
5.3.2.1	Convergence Behavior	65
5.3.3	Test Set Performance	66
5.3.3.1	Overall Metrics	66
5.3.3.2	Per-Class Performance	67
5.3.3.3	Confusion Matrix	68
5.4	Discussion and Insights	68
5.4.1	Why ResNet50 for This Problem?	68
5.4.2	Data vs Architecture Trade-offs	69
5.4.3	Lessons Learned	69
5.5	Technical Implementation Details	70
5.5.1	Code Architecture	70
5.5.2	Framework and Versions	71
5.5.3	Reproducibility Considerations	71
5.6	Conclusions and Future Directions	71
5.6.1	Summary	71
5.6.2	Key Findings	72
5.6.3	Future Directions	72
	Summary and Future Work	74
	A Additional Implementation Details	76
A.1	Model Training Details	76
A.2	Computational Resources	76
	Bibliography	77

List of Figures

1.1	Representative polished steel microstructure showing non-metallic inclusions.	2
1.2	Sulfide inclusion types and morphology examples.	3
1.3	Fatigue strength optimization through rare earth element modification.	5
1.4	Polished microstructure samples 1 and 2 showing various inclusion morphologies.	7
1.5	Polished microstructure samples 3 and 4 illustrating inclusion variations.	8
1.6	Polished microstructure samples 5 and 6 from different steel pieces.	8
3.1	U-Net architecture diagram showing encoder-decoder structure with skip connections.	17
3.2	U-Net prediction results on test micrograph.	21
3.3	U-Net++ prediction on test micrograph.	24
3.4	Attention U-Net mechanism illustration.	25
3.5	Attention U-Net prediction on test micrograph.	29
3.6	ENet architecture showing efficient design principles.	30
3.7	ENet prediction on test micrograph.	34
3.8	Mask R-CNN two-stage detection pipeline.	36
3.9	Mask R-CNN prediction showing instance-level masks on test micrograph.	39
5.1	Transfer learning pipeline: pre-training on ImageNet followed by fine-tuning on the steel inclusion dataset.	45
5.2	Example illustration of 2D convolution operation sliding a kernel over an image to produce feature maps.	51
5.3	ResNet50 architecture with residual bottleneck blocks and skip connections.	54
5.4	Overview of the deep learning model architecture.	60
5.5	Custom multi-label classifier head architecture.	61
5.6	Distribution of inclusion classes in the dataset.	64
5.7	Training and validation loss and F1 curves showing convergence and onset of overfitting.	65
5.8	Sample predictions on the test set.	66
5.9	Per-class precision, recall, and F1 scores for inclusion types.	67

5.10	Confusion matrix summarizing true vs predicted labels per inclusion class.	68
5.11	Relationship between training class frequency and test F1 scores highlighting class imbalance impact.	70

List of Tables

2.1	Quantitative comparison of segmentation models.	12
5.1	Dataset composition and class-wise distribution	64
5.2	Training and validation metrics at epochs 20 and 30	65
5.3	Summary performance metrics on the test set	66
5.4	Per-class metrics on the test set	67

Abbreviations

CNN Convolutional Neural Network

PA Pixel Accuracy

F1 F1-score

ASTM American Society for Testing and Materials

EDS Energy Dispersive Spectroscopy

SEM Scanning Electron Microscopy

RPN Region Proposal Network

RoI Region of Interest

Symbols

Al_2O_3	Aluminum Oxide (Alumina)	—
MnS	Manganese Sulfide	—
SiO_2	Silicon Dioxide (Silica)	—
σ	Stress Tensor	—
ε	Strain Tensor	—

Chapter 1

Theory of Inclusions and Metallographic Basis

1.1 Non-metallic inclusions in steel

Non-metallic inclusions are chemical compounds of metallic elements (Fe, Mn, Al, Si, Ca) with non-metals such as O, S, C, H, and N, forming separate phases embedded within the steel matrix. They typically arise during steelmaking due to deoxidation, desulfurization, reoxidation, or contamination from refractories and slag.

These inclusions are generally undesirable beyond controlled levels, because they can act as crack initiation sites, reduce toughness and fatigue strength, and affect surface quality and machinability. Their impact depends strongly on type, size, shape, distribution, and volume fraction.

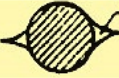
Behaviour of inclusions during deformation			
Type	Description	Before rolling	After rolling
(a)	A hard inclusion under rolling conditions		
(b)	A hard crystalline inclusion broken during rolling		
(c)	A hard inclusion cluster strung out during rolling		
(d)	An inclusion composed of hard crystals dispersed in a soft matrix		
(e)	A soft inclusion under rolling conditions		

FIGURE 1.1: Representative polished steel microstructure showing non-metallic inclusions.

1.2 Classification of Inclusions

Inclusions are generally classified based on their origin and chemical composition.

1.2.1 Origins

Indigenous (Endogenous) Inclusions: These form as a result of chemical reactions within the molten steel, such as deoxidation (removal of oxygen) or desulfurization. They are typically small and widely distributed. Examples include alumina (Al_2O_3) formed during aluminum deoxidation.

Exogenous Inclusions: These result from the mechanical entrapment of external materials, such as refractory particles from the furnace lining, ladle, or slag. These are often larger, irregularly shaped, and more detrimental to mechanical integrity.

1.2.2 Morphological Classification (ASTM E45)

For this project, we focused on the standard morphological types defined by ASTM standards:

1.2.2.1 Sulfide Inclusions (Type A)

Composition: Primarily Manganese Sulfide (MnS).

Morphology: Highly malleable. During hot working (rolling/forging), they deform plastically, resulting in long, thin, gray stringers with rounded ends.

Visual Characteristic: Light gray in reflected light, often appearing as elongated lines.

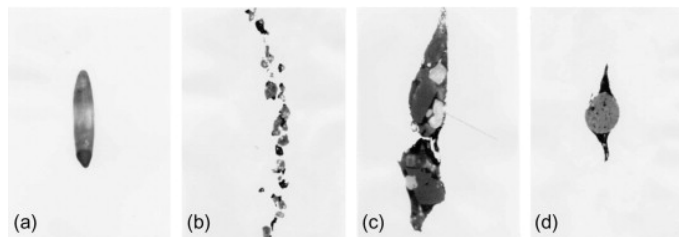


FIGURE 1.2: Sulfide inclusion types and morphology examples.

1.2.2.2 Alumina Inclusions (Type B)

Composition: Aluminum Oxide (Al_2O_3).

Morphology: Brittle and hard. They do not deform during hot working; instead, they fracture into chains of small, angular, non-deformable particles.

Visual Characteristic: Dark/black, angular particles arranged in a discontinuous track (stringer) parallel to the rolling direction.

1.2.2.3 Silicate Inclusions (Type C)

Composition: Silicates (SiO_2 -based).

Morphology: Deformable but less so than sulfides. They elongate into long stringers but tend to have sharper, more pointed ends compared to sulfides.

Visual Characteristic: Dark gray or black continuous stringers.

1.2.2.4 Globular Oxide Inclusions (Type D)

Composition: Oxides that do not fit into Type B (e.g., Calcium aluminates).

Morphology: Non-deformable, spherical or globular shapes. They generally remain randomly distributed and do not form stringers.

Visual Characteristic: Circular, dark particles.

1.3 Problems Caused by Inclusions

Inclusions disrupt the homogeneity of the steel matrix, acting as stress concentration risers.

Fatigue Failure: In high-strength steels, hard inclusions (like Alumina) often serve as crack initiation sites under cyclic loading, significantly reducing fatigue life.

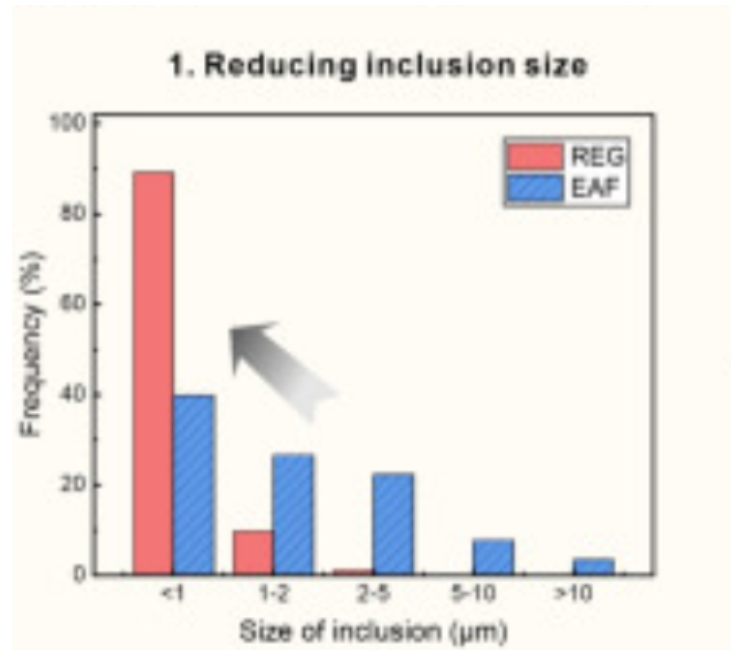


FIGURE 1.3: Fatigue strength optimization through rare earth element modification.

Reduced Ductility and Toughness: Elongated inclusions (Sulfides) cause anisotropy, where mechanical properties differ in the longitudinal versus transverse directions. Transverse ductility is often severely reduced.

Poor Surface Finish: In manufacturing, inclusions can lead to surface defects during polishing or plating, leading to rejected parts in automotive or aerospace applications.

Nozzle Clogging: During continuous casting, solid alumina inclusions can clog submerged entry nozzles, interrupting production.

1.4 Methods of detecting and characterizing inclusions

Traditional inclusion assessment relies on metallography and microscopy:

Specimen preparation: Cutting, mounting, grinding, and polishing to obtain a flat, scratch-free surface suitable for optical or electron microscopy.

Microscopic examination: Performed in the as-polished condition to observe inclusion size, shape, and distribution; for inclusion rating, etching is often avoided to prevent additional features from obscuring inclusions.

Scanning Electron Microscopy (SEM): Uses electron beams to image inclusions at higher magnification.

Energy Dispersive Spectroscopy (EDS): Often coupled with SEM to determine the elemental composition (e.g., confirming Al and O in an alumina inclusion).

In this project, high-resolution optical micrographs of polished steel samples were acquired and used as input for both segmentation and classification models, mirroring the data produced in industrial metallography.

1.5 ASTM Standards

The project adheres to principles outlined in key international standards:

ASTM E45: *Standard Test Methods for Determining the Inclusion Content of Steel.* This standard defines the microscopic diagnostic methods (Types A, B, C, D) and rating charts used to quantify inclusion severity.

ASTM E1122 (Withdrawn/Replaced): Historically dealt with obtaining JK inclusion ratings using automatic image analysis.

ASTM E1245: *Standard Practice for Determining the Inclusion or Second-Phase Constituent Content of Metals by Automatic Image Analysis.* This standard is crucial for modern automated systems.

ASTM E2142: Extends inclusion rating to automated SEM/EDX-based methods, allowing classification using chemical composition in addition to morphology.

1.6 Sample preparation and polishing in this project

To build a realistic dataset for inclusion classification, steel samples were sectioned from 5–6 different pieces, then mounted and progressively ground and polished to obtain a mirror-like finish suitable for high-magnification imaging. Proper polishing removes scratches and deformation layers that could hide or mimic inclusions, thereby improving the reliability of downstream image analysis. Approximately 107 optical micrographs were captured at suitable magnifications, capturing a range of inclusion morphologies; these images formed the base dataset for the classification stage.

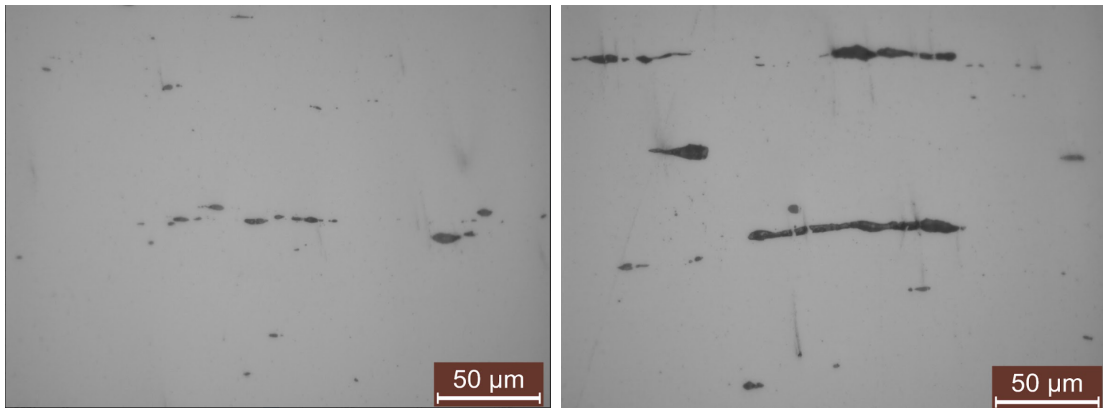


FIGURE 1.4: Polished microstructure samples 1 and 2 showing various inclusion morphologies.

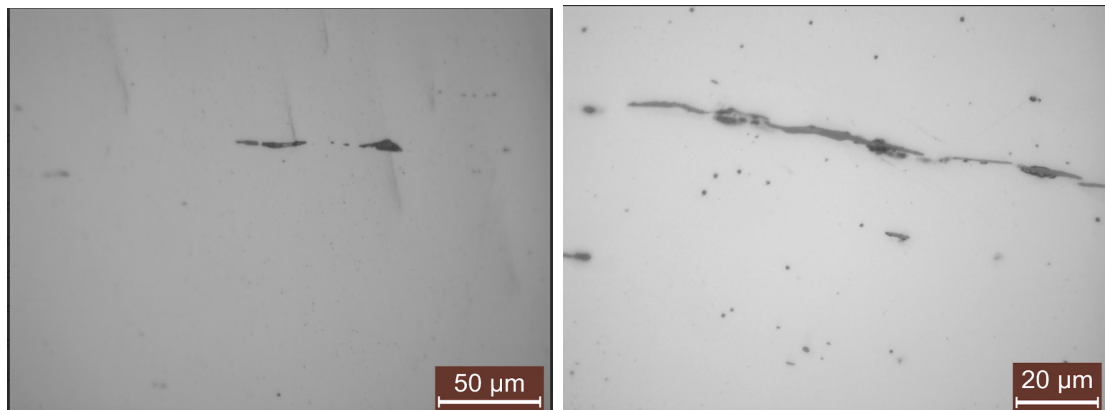


FIGURE 1.5: Polished microstructure samples 3 and 4 illustrating inclusion variations.

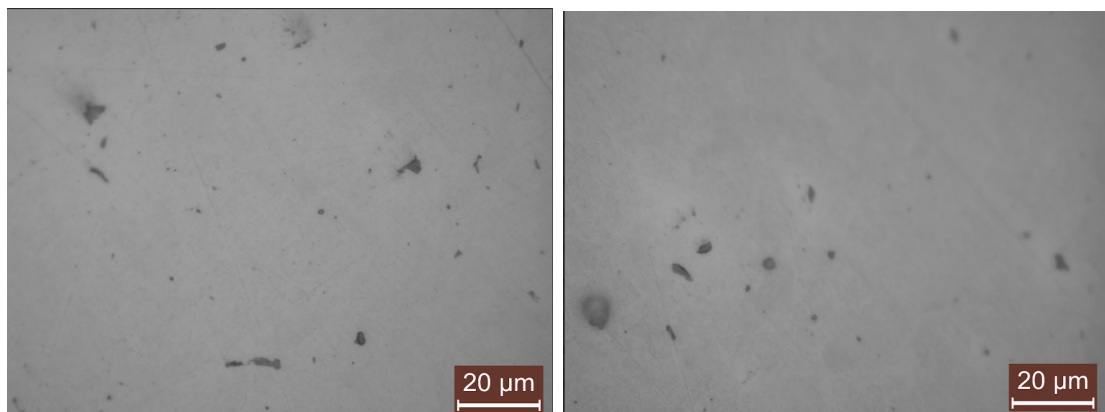


FIGURE 1.6: Polished microstructure samples 5 and 6 from different steel pieces.

Chapter 2

Segmentation Models and Results

2.1 Objective

The first phase of the project focused on **Inclusion Prediction** (Segmentation). The goal was to accurately identify pixels belonging to inclusions within a micrograph, effectively separating defects from the steel matrix.

2.2 Dataset acquisition and augmentation for segmentation

A dataset of 40–50 raw micrographs was sourced online. The “Roboflow” platform was utilized for manual annotation. Inclusions were labeled at the pixel level (polygons) to create segmentation masks. To overcome the limitation of a small dataset, aggressive data augmentation was applied using Roboflow. Techniques included:

- Rotations (90° , 180° , arbitrary angles).
- Flipping (Horizontal/Vertical).

- Shear transformations.

This augmentation expanded the dataset, improving model generalization and robustness. The final dataset was exported in the **COCO Segmentation** format, which provides bounding boxes and segmentation masks for each object.

2.3 COCO-format annotations and data pipeline

The processed dataset was exported from Roboflow in COCO segmentation format, allowing the use of standard PyTorch and torchvision utilities. A custom `CocoMaskDataset` class derived from `torchvision.datasets.CocoDetection` was implemented to load images and convert polygon annotations into binary masks for each inclusion instance. For U-Net-style models that expect image-mask pairs, a separate routine converted COCO polygon masks into per-pixel mask images (PNG) and stored them in train/validation/test “masks” folders, enabling straightforward loading via a custom `SegmentationDataset` wrapper.

2.4 Overview of evaluated segmentation architectures

Five architectures were implemented and trained on the same dataset to provide a comparative study:

2.4.1 U-Net

A classic encoder-decoder architecture originally designed for biomedical image segmentation. It uses skip connections to concatenate features from the contracting path to the expansive path, preserving spatial information.

2.4.2 U-Net++

A nested U-Net architecture with dense skip pathways. It bridges the semantic gap between encoder and decoder feature maps, often improving segmentation accuracy for objects of varying scales.

2.4.3 Attention U-Net

Integrates “Attention Gates” into the U-Net skip connections. These gates learn to suppress irrelevant regions (the steel matrix) and highlight salient features (inclusions) without additional supervision.

2.4.4 ENet (Efficient Neural Network)

A lightweight encoder-decoder architecture optimized for real-time processing. It sacrifices some depth for speed, making it suitable for resource-constrained environments.

2.4.5 Mask R-CNN

A two-stage detector. *Stage 1*: Region Proposal Network (RPN) generates candidate bounding boxes. *Stage 2*: Predicts the class, box offset, and a binary mask for each ROI in parallel. It is known for high accuracy in instance segmentation.

All U-Net variants and ENet used a common training framework with BCE loss, Adam optimizer (learning rate = 1×10^{-4}), and 20 training epochs, while Mask R-CNN used the standard detection training loop with SGD, momentum, and a smaller number of epochs initially.

2.5 Data loaders, training configuration, and metrics

For the U-Net-style models, images and masks were resized to a fixed size and converted to tensors, with batch size 4 for both training and validation. Binary cross-entropy loss on the predicted probability maps encouraged accurate estimation of inclusion versus background at each pixel, and learning curves showed consistent reduction in training loss over 20 epochs for all models. To evaluate performance, two metrics were computed on the test set:

- **Pixel Accuracy (PA)**: defined as the ratio of correctly classified pixels to total pixels.
- **Dice Coefficient**: defined as $\frac{2 \times TP}{2 \times TP + FP + FN}$, which is sensitive to class imbalance and is widely used in segmentation tasks.

2.6 Quantitative segmentation results

TABLE 2.1: Quantitative comparison of segmentation models.

Model	Pixel Accuracy (PA)	Dice Coefficient
U-Net	0.9581	0.7488
U-Net++	0.9514	0.7347
Attention U-Net	0.9559	0.7450
ENet	0.9428	0.6644
Mask R-CNN	0.9630	0.7974

Mask R-CNN achieved the highest pixel accuracy and Dice score, suggesting that instance-level modeling and a strong ResNet-FPN backbone were beneficial even with limited data. U-Net and Attention U-Net performed similarly, with Attention

U-Net slightly improving Dice, while U-Net++ and ENet trailed behind, likely due to increased optimization complexity (U-Net++) and model capacity constraints (ENet).

2.7 Analysis of Results

Best Performer: **Mask R-CNN** outperformed all other models with a Pixel Accuracy of 96.30% and a Dice score of 0.7974.

Reasoning: Inclusions often appear as distinct, separate instances (particles or stringers). Mask R-CNN is inherently an instance segmentation model, which treats each inclusion as a separate object. This architectural bias aligns well with the physical nature of inclusions. Furthermore, the Region Proposal Network (RPN) effectively filters out the large background (clean steel), focusing the segmentation head only on relevant areas.

U-Net Variants: While U-Net and Attention U-Net performed well (approx 95.8% PA), they occasionally struggled with the boundaries of very fine sulfide stringers, leading to slightly lower Dice scores.

ENet: While the fastest, ENet provided the lowest accuracy (66% Dice), indicating it may lack the capacity to capture the fine morphological details required for precise inclusion delineation.

Chapter 3

Detailed Analysis of Segmentation Models

3.1 Common Building Blocks Used in All CNN Models

3.1.1 Convolution

A convolution operation between an input feature map X and kernel W is defined as:

$$Y(i, j) = \sum_m \sum_n X(i - m, j - n) W(m, n).$$

3.1.2 Batch Normalization

Batch normalization normalizes activations channel-wise:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}},$$

$$y = \gamma \hat{x} + \beta,$$

where μ, σ^2 are batch statistics and γ, β are learnable parameters.

3.1.3 Activation Functions

ReLU:

$$\text{ReLU}(x) = \max(0, x).$$

PReLU:

$$\text{PReLU}(x) = \begin{cases} x, & x \geq 0, \\ ax, & x < 0, \end{cases}$$

where a is a learnable parameter.

3.1.4 Pooling

Max pooling:

$$Y(i, j) = \max_{(m, n) \in \Omega} X(i + m, j + n).$$

Average pooling:

$$Y(i, j) = \frac{1}{|\Omega|} \sum_{(m, n) \in \Omega} X(i + m, j + n).$$

3.1.5 5. Convolution Transpose (Deconvolution)

The transposed convolution for upsampling is defined as:

$$Y = W^T * X.$$

3.1.6 Dropout

Dropout randomly zeros activations during training:

$$\tilde{x}_i = \begin{cases} 0, & \text{with probability } p, \\ \frac{x_i}{1-p}, & \text{with probability } 1-p. \end{cases}$$

3.1.7 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

3.1.8 Softmax

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

3.1.9 Binary Cross-Entropy Loss

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

3.2 U-Net

U-Net is a kind of neural network mainly used for image segmentation, which means dividing an image into different parts to identify specific objects. The architecture is symmetric and has three key parts:

3.2.1 Architecture Components

1. Contracting Path (Encoder):

- Uses small filters (3×3 pixels) to scan the image and find features.
- Applies an activation function called ReLU to add non-linearity and help the model learn better.
- Uses max pooling (2×2 filters) to shrink the image size while keeping important information. This helps the network focus on bigger features.

2. Bottleneck: The middle of the “U” where the most compressed and abstract information is stored. It links the encoder and decoder.

3. Expansive Path (Decoder):

- Uses upsampling (increasing image size to get back the original image size).
- Combines information from the encoder using “skip connections.” These connections help the decoder get spatial details that might have been lost when shrinking the image.
- Uses convolution layers again to clean up and refine the output.

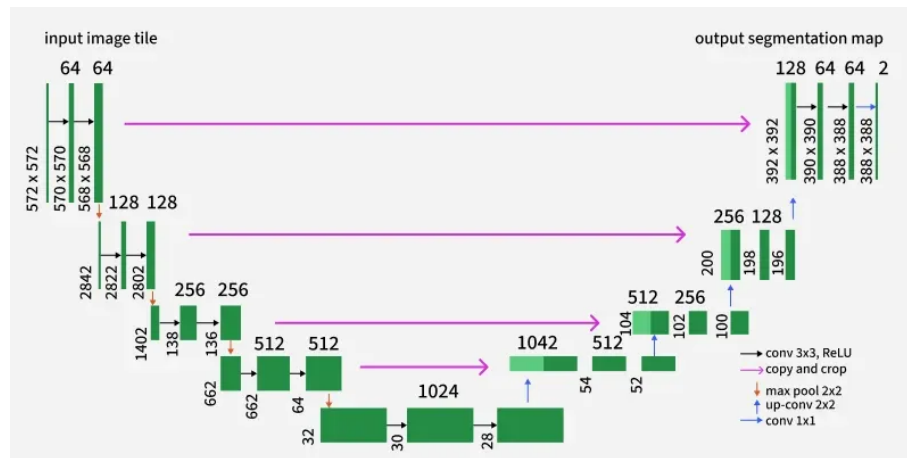


FIGURE 3.1: U-Net architecture diagram showing encoder-decoder structure with skip connections.

3.2.2 How U-Net Works

After understanding the architecture, the U-Net processing pipeline is as follows:

Input Image: The process starts by feeding an input image (typically grayscale) into the network.

Feature Extraction (Encoder): The encoder extracts increasingly abstract features by applying convolutions and downsampling. At each level, the spatial size decreases while the number of feature channels increases, allowing the model to capture higher-level patterns.

Bottleneck Processing: This is the middle part of the network where the image is reduced the most. It holds a small but very meaningful version of the image that captures the main features.

Reconstruction and Localization (Decoder): The decoder begins to reconstruct the original image size through upsampling. At each level, it combines decoder features with corresponding encoder features using skip connections to retain fine-grained spatial details.

Skip Connections for Precision: Skip connections help preserve spatial accuracy by bringing forward detailed features from earlier layers. These are especially useful when the model needs to distinguish boundaries in segmentation tasks.

Final Prediction: A 1×1 convolution at the end converts the refined feature maps into the final segmentation map where each pixel is classified into a specific class like foreground or background. This output has the same spatial resolution as the input image.

3.2.3 Mathematical Formulation

Let the input image be $X^0 \in \mathbb{R}^{C \times H \times W}$.

Each U-Net encoder block consists of two sequential convolutions (Conv + ReLU):

$$\text{ConvBlock}(X) = \phi(W_2 * \phi(W_1 * X + b_1) + b_2),$$

where ϕ is ReLU.

For encoder depth levels $l = 1, 2, \dots, L$:

$$E^l = \text{ConvBlock}(E^{l-1})$$

Downsampling via 2×2 max pooling:

$$D^l = \text{MaxPool}(E^l)$$

The encoder outputs feature maps:

$$E^0, E^1, \dots, E^L$$

Each decoder stage first upsamples the previous decoder output:

$$U^l = \text{Up}(Z^{l+1})$$

Skip connection concatenation:

$$C^l = [U^l, E^l]$$

Then a ConvBlock:

$$Z^l = \text{ConvBlock}(C^l)$$

The decoder produces:

$$Z^{L-1}, Z^{L-2}, \dots, Z^0$$

A 1×1 convolution maps decoder output to a single mask channel:

$$S = W_{\text{out}} * Z^0 + b_{\text{out}}$$

Binary segmentation uses a sigmoid activation:

$$\hat{Y} = \sigma(S)$$

$$E^l = \text{ConvBlock}(D^{l-1}), \quad l = 1, \dots, L$$

$$D^l = \text{MaxPool}(E^l),$$

$$U^l = \text{Up}(Z^{l+1}),$$

$$C^l = [U^l, E^l],$$

$$Z^l = \text{ConvBlock}(C^l),$$

$$\hat{Y} = \sigma(W_{\text{out}} * Z^0 + b_{\text{out}}).$$

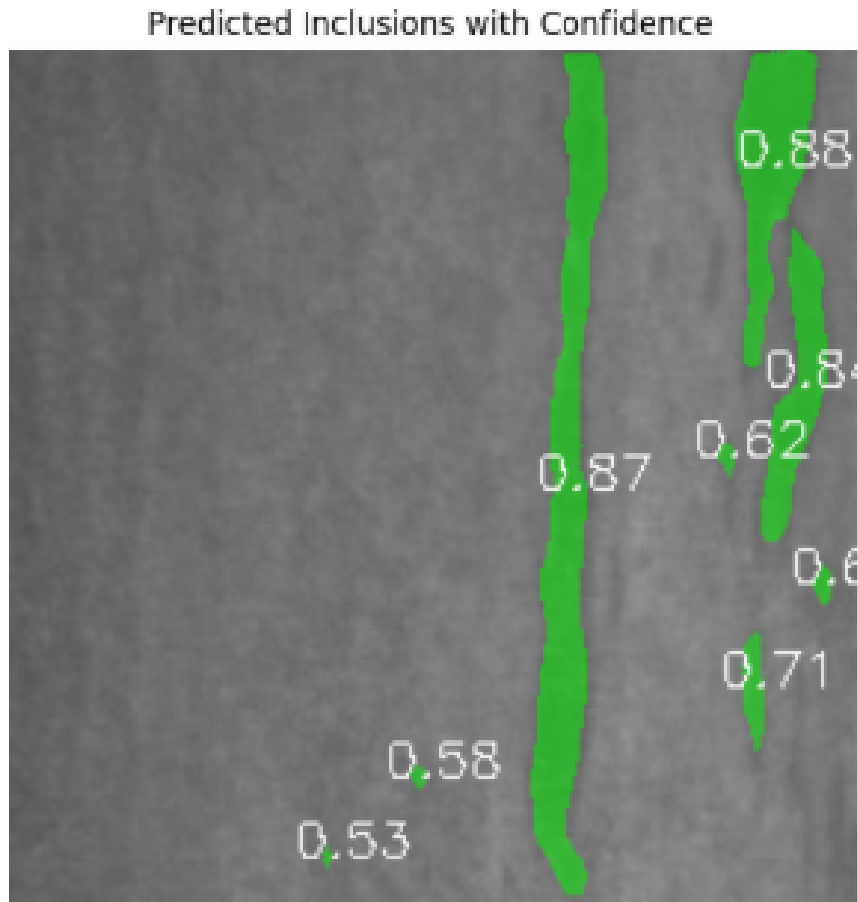


FIGURE 3.2: U-Net prediction results on test micrograph.

3.3 U-Net++

U-Net++ architecture is a semantic segmentation architecture based on U-Net. It introduces two main innovations in the traditional U-Net architecture, namely, nested dense skip connections and deep supervision. The authors found that using nested dense skip connections bridges the semantic gap between encoder and decoder feature maps and improves the gradient flow. They also found that using deep supervision enhances the model performance by providing a regularization to the network while training.

3.3.1 Key Improvements

Nested Skip Connections: U-Net++ architecture uses a nested skip connection network to aggregate the features while decoding the encoded data. By aggregating these features from different paths in the network, the model is able to improve the accuracy of the segmentation mask.

Deep Supervision: U-Net++ architecture uses deep supervision to enhance the model performance by providing a regularization to the network while training.

3.3.2 Mathematical Formulation

U-Net++ introduces *nested dense skip connections*. Each node $X^{i,j}$ represents the feature map at depth i and dense level j .

Encoder ($j = 0$ nodes) The base encoder nodes are:

$$X^{0,0} = \text{ConvBlock}(X)$$

For deeper levels:

$$X^{i,0} = \text{ConvBlock}(\text{Down}(X^{i-1,0})), \quad i = 1, 2, \dots, L$$

Decoder: Nested Dense Skip Connections ($j \geq 1$)

For each node $X^{i,j}$ where $j \geq 1$:

1. ****Upsampling from the node below:****

$$U^{i,j} = \text{Up}(X^{i+1,j-1})$$

2. ****Concatenation of all previous nodes along the skip path:****

$$C^{i,j} = [X^{i,0}, X^{i,1}, \dots, X^{i,j-1}, U^{i,j}]$$

3. ****Apply convolution block:****

$$X^{i,j} = \text{ConvBlock}(C^{i,j})$$

U-Net++ Dense Skip Path Summary Formally:

$$X^{i,j} = \text{ConvBlock}([X^{i,0}, X^{i,1}, \dots, X^{i,j-1}, \text{Up}(X^{i+1,j-1})])$$

For all valid i, j :

$$i \in \{0, 1, \dots, L-1\}, \quad j \in \{1, 2, \dots, D\}$$

Final Output

U-Net++ uses the full-resolution node:

$$\hat{Y} = \sigma(W_{\text{out}} * X^{0,D} + b_{\text{out}})$$

Complete Forward Equations

$$X^{i,0} = \text{ConvBlock}(\text{Down}(X^{i-1,0})),$$

$$U^{i,j} = \text{Up}(X^{i+1,j-1}),$$

$$C^{i,j} = [X^{i,0}, X^{i,1}, \dots, X^{i,j-1}, U^{i,j}],$$

$$X^{i,j} = \text{ConvBlock}(C^{i,j}),$$

$$\hat{Y} = \sigma(W_{\text{out}} * X^{0,D} + b_{\text{out}}).$$

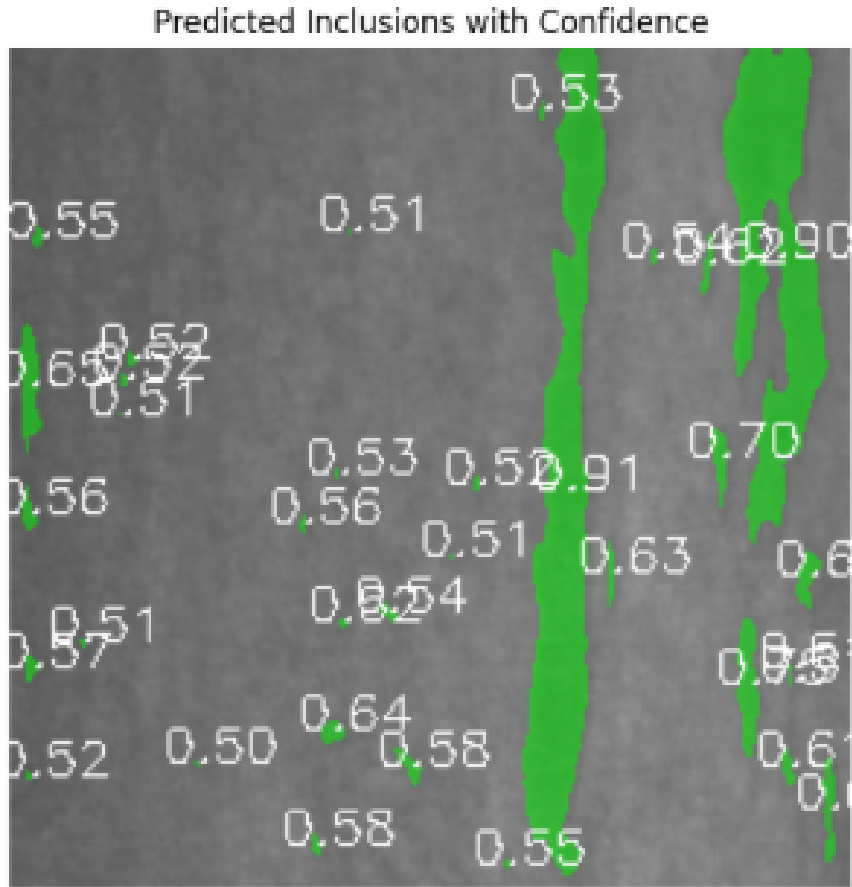


FIGURE 3.3: U-Net++ prediction on test micrograph.

3.4 Attention U-Net

Attention is an important aspect of any computer vision task where we need to focus on specific regions in an image. Regarding image segmentation, attention is a massive help in highlighting the important regions of the image. Due to this, the model converges much faster and has better generalization capabilities. The models trained with Attention Gate implicitly learn to suppress irrelevant regions in an input image while highlighting salient features useful for a specific task. This enables us to eliminate the necessity of using any explicit localization modules. The Attention Gate can be easily integrated into any existing segmentation architecture such as U-Net with a minimal increase in the parameters. This increase in parameters boosts the model's sensitivity and accuracy.

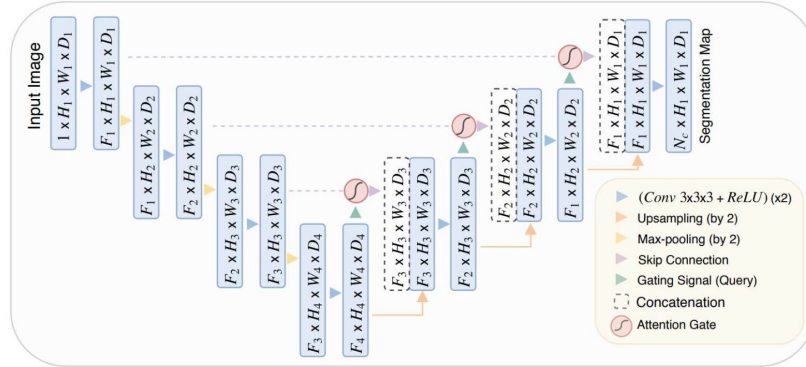


FIGURE 3.4: Attention U-Net mechanism illustration.

3.4.1 Mathematical Formulation

Attention U-Net enhances the standard U-Net by introducing *attention gates (AGs)* that suppress irrelevant skip features while highlighting task-relevant activations.

Each attention gate takes:

$$x^l \in \mathbb{R}^{C_x \times H \times W} \quad (\text{skip connection input})$$

$$g \in \mathbb{R}^{C_g \times H' \times W'} \quad (\text{decoder gating signal})$$

ear Transformations

The skip feature and gating signal are first linearly projected:

$$x' = W_x * x^l + b_x, \quad g' = W_g * g + b_g$$

where: - W_x is a 1×1 convolution mapping $C_x \rightarrow F$, - W_g is a 1×1 convolution mapping $C_g \rightarrow F$.

After spatial alignment (upsampling if needed), we compute:

$$\eta = x' + g'$$

Linear Transformations

Intermediate Attention Coefficients

A nonlinearity is applied:

$$\beta = \sigma_1(\eta)$$

where σ_1 is usually a ReLU:

$$\sigma_1(z) = \max(0, z)$$

Then another 1×1 convolution reduces to a single-channel attention coefficient map:

$$\alpha = \sigma_2(W_\psi * \beta + b_\psi)$$

where σ_2 is the sigmoid function:

$$\sigma_2(z) = \frac{1}{1 + e^{-z}}$$

Attention-weighted Skip Features

The skip connection is filtered by the learned attention map:

$$\tilde{x}^l = \alpha \cdot x^l$$

which is sent to the decoder via concatenation:

$$Z^l = \text{ConvBlock}([\tilde{x}^l, \text{Up}(Z^{l+1})])$$

Final Output Layer

After the last decoder stage:

$$S = W_{\text{out}} * Z^0 + b_{\text{out}}$$

Binary segmentation output:

$$\hat{Y} = \sigma(S)$$

- Com-

plete Forward Equations Summary

$$x' = W_x * x^l + b_x,$$

$$g' = W_g * g + b_g,$$

$$\eta = x' + g',$$

$$\beta = \sigma_1(\eta),$$

$$\alpha = \sigma_2(W_\psi * \beta + b_\psi),$$

$$\tilde{x}^l = \alpha \cdot x^l,$$

$$Z^l = \text{ConvBlock}([\tilde{x}^l, \text{Up}(Z^{l+1})]),$$

$$\hat{Y} = \sigma(W_{\text{out}} * Z^0 + b_{\text{out}}).$$

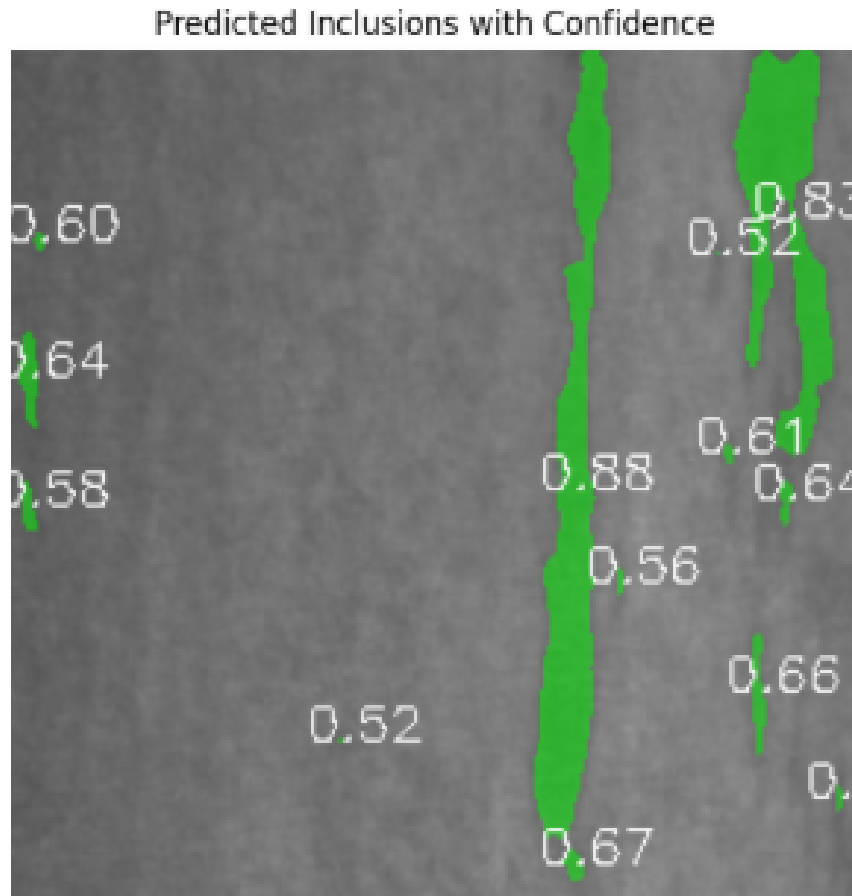


FIGURE 3.5: Attention U-Net prediction on test micrograph.

3.5 ENet (Efficient Neural Network)

ENet is an efficient segmentation architecture optimized for speed and low memory consumption. It employs several key techniques:

3.5.1 Architectural Features

Early Downsampling for Speed: ENet aggressively downsamples the input early in the network (using strided convolutions and pooling), drastically reducing feature-map size and making the model extremely fast and lightweight.

Factorized and Asymmetric Convolutions: It uses asymmetric convolutions (e.g., 5×1 followed by 1×5) and factorized operations to reduce computation while keeping good receptive field coverage.

Dilated Convolutions for Larger Context: ENet applies dilated convolutions in the middle encoder blocks to expand the receptive field without increasing parameters, helping detect larger structures.

Bottleneck Modules: Most layers are bottleneck blocks ($1 \times 1 \rightarrow \text{conv/dilated/asymmetric} \rightarrow 1 \times 1$), which compress and then expand features, keeping the model very efficient.

Lightweight Decoder: The decoder is intentionally small and uses upsampling plus few convolutions, focusing on restoring spatial resolution without heavy computation. Most learning happens in the encoder.

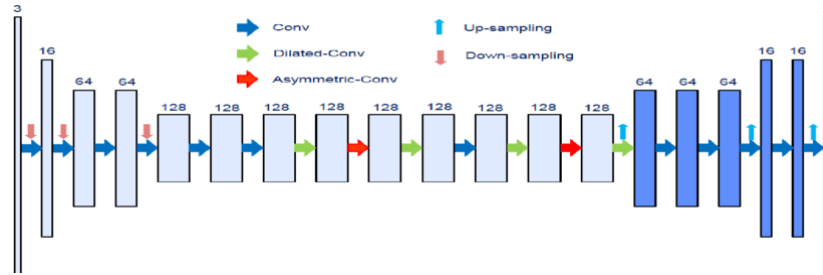


FIGURE 3.6: ENet architecture showing efficient design principles.

3.5.2 Mathematical Formulation

Initial Block

The ENet initial block combines a convolution branch and a pooling branch, concatenated along the channel dimension:

$$Y = \text{Concat}\left(\text{Conv}_{3 \times 3, s=2}(X), \text{Pool}_{2 \times 2}(X)\right)$$

Batch normalization and PReLU activation follow:

$$Z = \text{PReLU}(\text{BN}(Y))$$

Regular Bottleneck (Encoder Block)

Each ENet bottleneck follows the sequence: 1. 1×1 projection 2. Main convolution (3×3 , dilated, or asymmetric) 3. 1×1 expansion 4. Residual addition

$$Y = \text{Conv}_{1 \times 1}^{\text{proj}}(X)$$

$$Y' = \text{Conv}_k(Y)$$

$$\hat{Y} = \text{Conv}_{1 \times 1}^{\text{expand}}(Y')$$

Final residual output:

$$Z = \text{PReLU}(X + \hat{Y})$$

Downsampling Bottleneck

The downsampling module applies strided convolution and max pooling in parallel:

$$Y_1 = \text{Conv}_{2 \times 2, s=2}(X)$$

$$Y_2 = \text{MaxPool}_{2 \times 2}(X)$$

Channel mismatch is corrected by zero-padding:

$$\tilde{Y}_2 = \text{Pad}(Y_2)$$

Concatenation and activation:

$$Z = \text{PReLU} \left(\text{BN} \left(\text{Concat}(Y_1, \tilde{Y}_2) \right) \right)$$

Dilated Convolution

ENet uses dilated convolutions to enlarge the receptive field without increasing parameters:

$$(Y *_r W)(i, j) = \sum_m \sum_n X(i - rm, j - rn) W(m, n)$$

where r is the dilation rate.

Asymmetric Convolution

ENet factorizes a 5×5 convolution into a 5×1 followed by a 1×5 :

$$\text{Conv}_{5 \times 5}(X) \approx \text{Conv}_{5 \times 1}(\text{Conv}_{1 \times 5}(X))$$

Decoder (Upsampling Bottleneck)

The upsampling bottleneck first reduces channels, then upsamples, then applies convolution:

$$Y = \text{Conv}_{1 \times 1}(X)$$

$$Y' = \text{Upsample}_{2 \times}(Y)$$

$$\hat{Y} = \text{Conv}_{3 \times 3}(Y')$$

The skip connection uses a transposed convolution:

$$S = \text{ConvTranspose}_{2 \times 2}(X)$$

Final output:

$$Z = \text{PReLU}(S + \hat{Y})$$

Final Segmentation Projection Layer

The final 1×1 convolution maps decoder output to class logits:

$$P = \text{Conv}_{1 \times 1}(Z)$$

Applying sigmoid for binary segmentation:

$$\hat{M} = \sigma(P)$$

Binary Cross-Entropy Loss

Training uses pixelwise binary cross-entropy:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

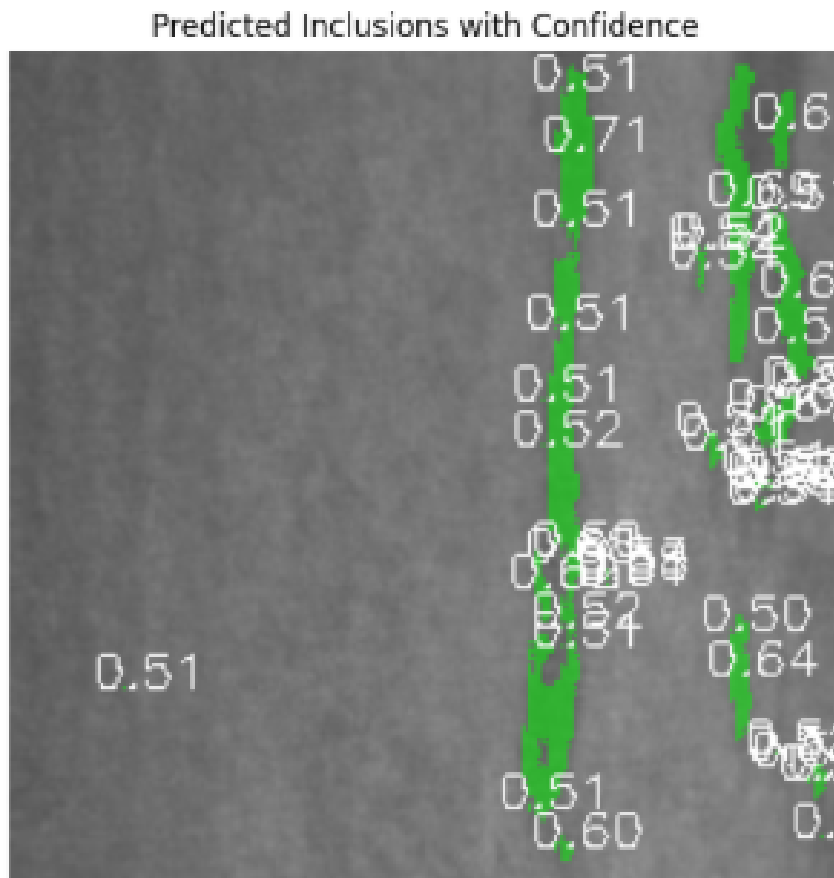


FIGURE 3.7: ENet prediction on test micrograph.

3.6 Mask R-CNN

The key idea behind R-CNN is to divide the object detection task into two stages: region proposal and object classification.

3.6.1 How Mask R-CNN Works

Region Proposal Network (RPN): In the first stage, the model generates a set of region proposals that are likely to contain objects. These proposals are potential bounding boxes around objects in the image. The region proposal network is responsible for suggesting these candidate regions.

Region of Interest (RoI) Pooling: Once the region proposals are generated, each region is cropped from the image and resized to a fixed size. This process is known as RoI pooling, and it ensures that the region of interest is consistently represented in a fixed-size feature map, regardless of the size or aspect ratio of the original region proposal.

Feature Extraction: The cropped and resized regions are then passed through a pre-trained convolutional neural network (CNN) to extract features from each region.

Object Classification and Bounding Box Regression: The features extracted from each region are used for two tasks: object classification and bounding box regression. Object classification involves determining the class of the object within the region, and bounding box regression refines the coordinates of the bounding box around the object.

Mask Prediction Head: Mask R-CNN extends Faster R-CNN with an additional branch for predicting a segmentation mask for each detected object.

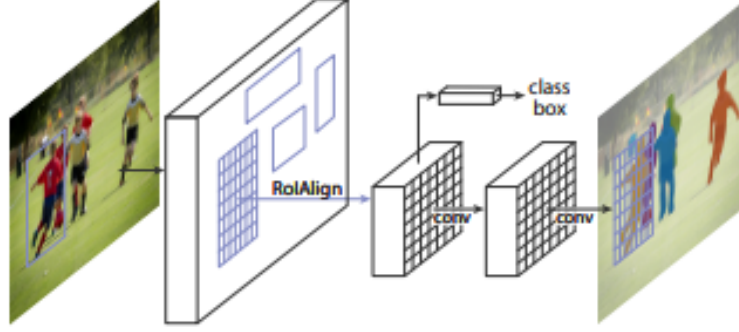


FIGURE 3.8: Mask R-CNN two-stage detection pipeline.

3.6.2 Mathematical Formulation

Region Proposal Network (RPN)

The RPN predicts objectness scores and bounding box offsets for anchor boxes.

Objective Score.

For each anchor box, the objectness logit is:

$$s = W_s * F + b_s,$$

where F is the feature map from the backbone.

The probability of being an object is obtained using the sigmoid activation:

$$p = \sigma(s).$$

Bounding Box Regression

The RPN regresses offsets (t_x, t_y, t_w, t_h) from anchor boxes:

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a},$$

$$t_w = \log\left(\frac{w}{w_a}\right), \quad t_h = \log\left(\frac{h}{h_a}\right),$$

where (x_a, y_a, w_a, h_a) are anchor coordinates.

RPN Loss.

$$\mathcal{L}_{RPN} = \mathcal{L}_{cls}(p, p^*) + \lambda \mathcal{L}_{reg}(t, t^*).$$

Classification loss (binary cross entropy):

$$\mathcal{L}_{cls} = -[p^* \log(p) + (1 - p^*) \log(1 - p)].$$

Smooth L1 regression loss:

$$\mathcal{L}_{reg}(t, t^*) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L1}(t_i - t_i^*).$$

RoI Align

RoI Align computes features at fractional pixel locations using bilinear interpolation:

$$F_{aligned}(x, y) = \sum_{i=1}^2 \sum_{j=1}^2 w_{ij} F(x_i, y_j),$$

where w_{ij} are bilinear interpolation weights.

Box Classification and Regression Head

Classification.

For C object classes:

$$p_c = \text{Softmax}(W_c f + b_c).$$

Bounding box Regression.

For each class:

$$\Delta t = W_{reg} f + b_{reg}.$$

Bounding box reconstruction:

$$\begin{aligned} \hat{x} &= x_a + t_x^* w_a, & \hat{y} &= y_a + t_y^* h_a, \\ \hat{w} &= w_a \exp(t_w^*), & \hat{h} &= h_a \exp(t_h^*). \end{aligned}$$

Loss.

$$\mathcal{L}_{box} = \mathcal{L}_{cls} + \mathcal{L}_{reg}.$$

Mask Head

For each class, the mask head predicts a pixelwise binary mask.

Let the predicted mask be:

$$\hat{M} = \sigma(W_m * F_{RoI} + b_m).$$

The mask loss (per class) uses pixelwise binary cross entropy:

$$\mathcal{L}_{mask} = -\frac{1}{N} \sum_{i=1}^N \left[M_i \log(\hat{M}_i) + (1 - M_i) \log(1 - \hat{M}_i) \right].$$

Total Mask R-CNN Loss

The total loss is a sum of all three heads:

$$\mathcal{L}_{total} = \mathcal{L}_{RPN} + \mathcal{L}_{box} + \mathcal{L}_{mask}.$$

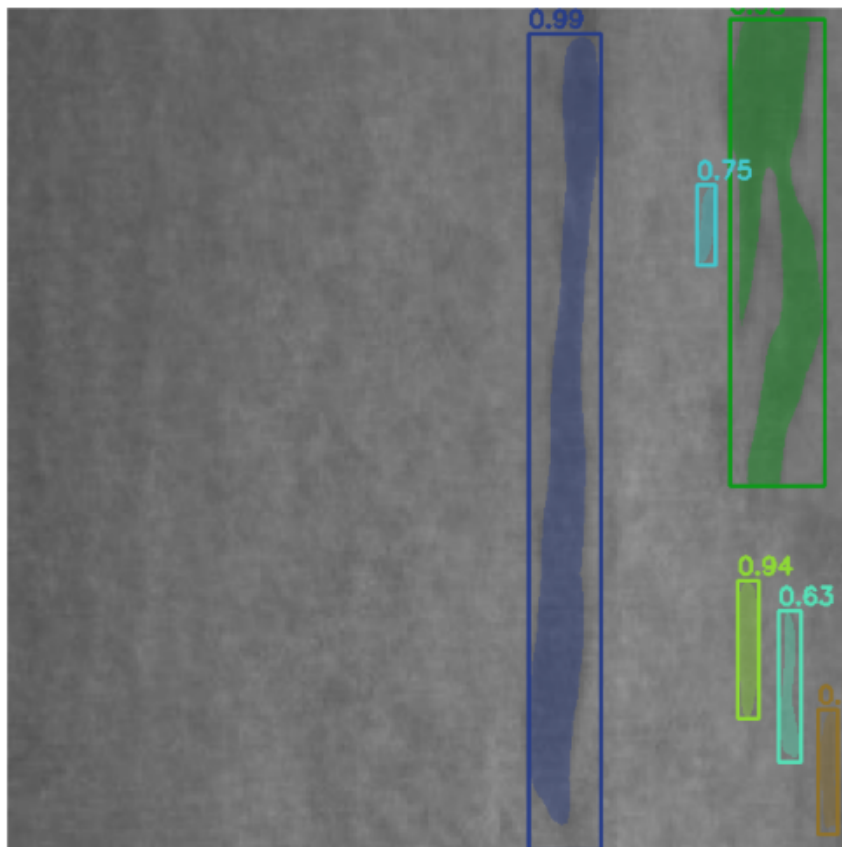


FIGURE 3.9: Mask R-CNN prediction showing instance-level masks on test micrograph.

Chapter 4

Need for Classification and Inclusion Types

4.1 Motivation for going beyond segmentation

Segmentation only identifies where inclusions are located but does not differentiate between different chemical or morphological types, which have different effects on material behavior and may originate from different process routes. In industrial quality control, knowing the type and frequency of each inclusion class is critical for diagnosing process problems (e.g., poor deoxidation, slag carryover) and for verifying compliance with standards like ASTM E45. The *type* of inclusion dictates the corrective action required in the steelmaking process:

- **High Alumina:** Indicates issues with deoxidation or reoxidation; requires checking aluminum killing practice or ladle shrouding.
- **High Sulfides:** Indicates a need for better desulfurization reagents (e.g., calcium treatment).

- **High Silicates:** Suggests exogenous sources like refractory erosion or improper slag separation.

Therefore, after successfully segmenting inclusion regions, this project extends the pipeline to classify inclusions into the four main classes: sulfide, alumina, silicate, and globular oxide, based on their appearance in polished micrographs.

4.2 Inclusion types for classification

The four label categories used in the classification network correspond to:

Sulfide Inclusion: Elongated or globular MnS-rich inclusions, often appearing as grey particles; they can severely affect fatigue and transverse properties if present as stringers.

Alumina Inclusion: Hard, non-deformable alumina particles arising from deoxidation or refractory erosion; they are typically angular and dark and can contribute to tool wear and spalling.

Silicate Inclusion: Complex silicates or spinels, sometimes elongated, that can form during solidification or from slag entrapment; these inclusions influence fatigue and machinability.

Globular Oxide Inclusion: More rounded oxides and oxy-sulfides that are often considered less harmful, representing well-modified inclusion populations. These categories align well with the type groups A–D in ASTM E45, making the output of the model directly interpretable in the context of standard inclusion rating.

4.3 Multi-label nature of the problem

In actual micrographs and in the prepared dataset, a single image can contain more than one inclusion type, leading naturally to a multi-label classification problem rather than a single-label setting. The dataset CSV exported from Roboflow contains binary indicators for each class per image, and many images are associated with combinations such as “alumina + sulfide” or “globular oxide + silicate.”

To reflect this, the classification pipeline treats each class as an independent binary label, using a sigmoid output per class and a threshold-based decision rule, allowing the model to predict any subset of the four inclusion types for a given micrograph.

4.4 Sample Preparation and Data Collection

To create a robust dataset for this phase:

Polishing: Steel samples were subjected to rigorous grinding (SiC papers) and polishing (diamond paste) to achieve a mirror finish, essential for revealing inclusion morphology.

Imaging: Approximately 107 high-resolution images were captured from 5–6 different steel samples using an optical microscope.

Labeling: Unlike the segmentation phase (masks), this phase involved **Multi-Label Classification**. A single image could contain multiple inclusion types (e.g., a Sulfide stringer alongside Globular Oxides).

Chapter 5

Inclusion Classification

5.1 Introduction

5.1.1 Motivation and Problem Statement

Steel quality is critically dependent on the control and detection of inclusions—foreign particles embedded within the steel matrix that degrade mechanical and corrosion-resistant properties. Inclusions are classified into four primary types based on their chemical composition and morphology:

- **Sulfide Inclusions (MnS , FeS):** Detrimental to ductility and fatigue strength.
- **Alumina Inclusions (Al_2O_3):** Harder oxides affecting surface quality.
- **Silicate Inclusions (SiO_2 , $\text{MgO} \cdot \text{SiO}_2$):** Form complex morphologies affecting properties.
- **Globular Oxide Inclusions:** Spherical oxides from metallurgical processes.

Traditional inclusion classification relies on manual microscopy examination, which is labour-intensive, subjective, and prone to human error and fatigue. Modern steel

plants produce hundreds of samples daily, making manual inspection economically impractical. Automated, objective detection systems are therefore essential for:

- **Real-time quality control:** Immediate feedback during production.
- **Consistent classification:** Eliminates observer bias.
- **Scalability:** Processes thousands of images efficiently.
- **Documentation:** Enables historical analysis and traceability.

5.1.2 Deep Learning Opportunity

Recent advances in deep learning, particularly convolutional neural networks (CNNs), have revolutionized computer vision. However, building high-performance models typically requires massive labelled datasets (ImageNet: 1.2M images). Steel inclusion classification datasets are considerably smaller (hundreds to thousands), presenting a constraint. **Transfer learning** addresses this constraint by leveraging pre-trained models on large datasets (ImageNet) and adapting them to specialized domains with minimal data. This approach has achieved state-of-the-art results across medical imaging, satellite imagery, and industrial inspection domains.

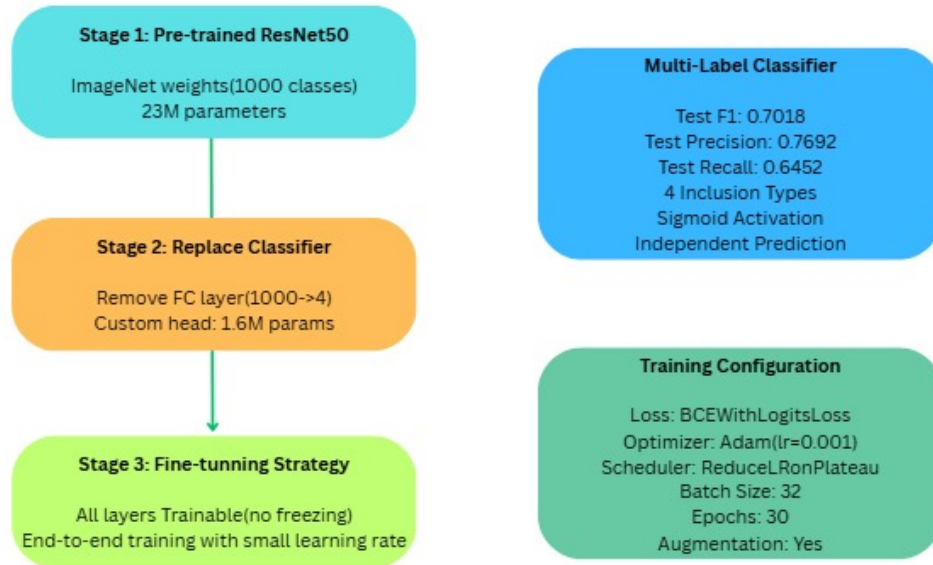


FIGURE 5.1: Transfer learning pipeline: pre-training on ImageNet followed by fine-tuning on the steel inclusion dataset.

5.1.3 Multi-Label Challenge

Unlike single-label classification (each image belongs to exactly one class), multi-label classification allows each image to belong to multiple classes simultaneously. Steel micrographs frequently contain multiple inclusion types, requiring explicit multi-label learning approaches. This introduces mathematical and algorithmic challenges not present in single-label settings.

5.1.4 Research Contributions

This section of the paper contributes:

- **Theoretical exposition** of transfer learning, CNNs, and multi-label classification for domain experts.

- **Complete implementation** of ResNet50-based multi-label classifier with architectural justifications.
- **Comprehensive data pipeline** including preprocessing, augmentation, and dataset conversion.
- **Empirical validation** demonstrating 10.3% F1-score improvement through data scaling.
- **Lessons learned** regarding data vs architecture trade-offs in specialized domains.

5.2 Theoretical Background

5.2.1 Machine Learning Fundamentals

5.2.1.1 Supervised Learning Paradigm

Machine learning can be formalized as finding a function that maps input space to output space using labelled examples:

$$f : \mathcal{X} \rightarrow \mathcal{Y}, \quad \text{where } x_i \in \mathcal{X}, y_i \in \mathcal{Y}.$$

Empirical Risk Minimization (ERM) principle states that the optimal function minimizes expected loss on the training distribution:

$$f^* = \arg \min_{f \in \mathcal{F}} \mathbb{E}_{(x,y) \sim \mathcal{D}} [L(f(x), y)],$$

where $L(\cdot)$ is the loss function and $\mathcal{F}$ belongs to some function class. In practice, we minimize empirical risk on finite data:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(f(x_i), y_i).$$

5.2.1.2 Classification vs Regression

Classification: Output space is discrete. Goal is to assign inputs to one or more classes.

- **Single-label classification:** Each sample belongs to exactly one class. Output space has exactly one 1. Loss: Cross-entropy loss penalizes probability on true class.
- **Multi-label classification:** Each sample may belong to multiple classes. Output space has arbitrary number of 1s. Loss: Binary cross-entropy loss treats each class independently.

Our problem is **multi-label image classification**: input space is RGB images, output space is four inclusion types.

5.2.1.3 Generalization and Overfitting

The fundamental ML challenge is **generalization**: model performs well on unseen data, not just training data. This requires balancing:

- **Model capacity:** Expressiveness of function class.
- **Training data size:** Number of examples.
- **Regularization:** Constraints preventing memorization.

Overfitting occurs when model memorizes training data but fails on test data. Deep neural networks have high capacity, making overfitting a critical concern. Regularization techniques include:

- L1/L2 weight decay: Penalize large weights.
- Dropout: Randomly zero activations during training.
- Early stopping: Stop training when validation performance plateaus.
- Data augmentation: Artificially expand training set.

5.2.2 Artificial Neural Networks

5.2.2.1 Neuron Model and Activation Functions

An artificial neuron computes a weighted sum of inputs followed by a nonlinear activation:

$$y = f(w \cdot x + b),$$

where w are weights, b is bias, and f is an activation function. **Activation functions** introduce non-linearity, allowing networks to learn complex decision boundaries:

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$. Output in $(0,1)$, suitable for binary classification.
- **Tanh**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Output in $(-1,1)$, more symmetric than sigmoid.
- **ReLU**: $\text{ReLU}(z) = \max(0, z)$. Non-negative, solves vanishing gradient problem. Computationally efficient and our choice.
- **Softmax**: $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Produces probability distribution over classes.

5.2.2.2 Multi-Layer Perceptrons (MLPs)

A neural network stacks multiple neuron layers:

$$\mathbf{y} = f_L(W_L \cdot f_{L-1}(W_{L-1} \cdots f_1(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_{L-1}) + \mathbf{b}_L).$$

Universal Approximation Theorem: A sufficiently wide MLP can approximate any continuous function on compact domains, given enough data and training. Limitation: MLPs require all inputs to be vectors, making them unsuitable for images. This motivated **Convolutional Neural Networks**.

5.2.2.3 Backpropagation Algorithm

Neural networks learn via **gradient descent**: iteratively update parameters in direction of negative gradient. **Backpropagation** efficiently computes gradients using the chain rule:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

For L layers, gradients propagate backwards:

$$\delta_l = (\delta_{l+1} W_{l+1}) \odot f'_l(z_l),$$

where $\odot$ denotes element-wise multiplication. **Complexity:** Forward pass is $O(L \cdot n^2)$ for L layers and n neurons per layer. Backward pass has same complexity. This quadratic scaling motivated structured architectures like CNNs.

5.2.3 Convolutional Neural Networks (CNNs)

5.2.3.1 Convolution Operation

CNNs exploit **spatial structure** in images. The convolution operation applies small learnable **kernels** (filters) across the image:

$$y[i, j] = \sum_{u, v} w[u, v] \cdot x[i + u, j + v] + b,$$

where x is input image, w is kernel, and output is the convolution at position (i, j) .

Advantages over fully-connected layers:

- **Locality:** Kernels capture local patterns (edges, corners, textures).
- **Parameter sharing:** Same kernel applied across image positions.
- **Translation invariance:** Learned patterns are robust to spatial shifts.
- **Reduced parameters:** For 256×256 image, fully-connected has $256^2 = 65K$ params per neuron. Convolution with 3×3 kernel has only 9 parameters.

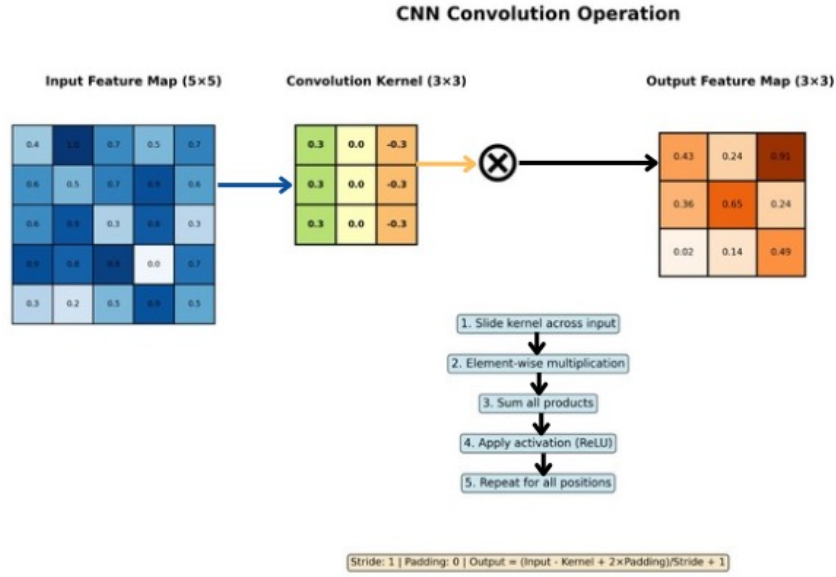


FIGURE 5.2: Example illustration of 2D convolution operation sliding a kernel over an image to produce feature maps.

5.2.3.2 Pooling Operations

Max pooling reduces spatial dimensions by taking maximum over local regions:

$$y[i, j] = \max_{u, v \in \text{pool}} x[i + u, j + v].$$

Benefits:

- Reduces feature map dimensions (and computation).
- Provides translation invariance.
- Highlights most salient activations.

5.2.3.3 Typical CNN Architecture

A standard CNN stacks convolution, activation, and pooling layers:

$$[\text{Conv}3\times3\text{-ReLU}] \rightarrow [\text{MaxPool}2\times2] \rightarrow [\text{Conv}3\times3\text{-ReLU}] \rightarrow [\text{MaxPool}2\times2] \rightarrow \dots$$

Progressive convolutions extract increasingly abstract features:

- Layer 1: Low-level features (edges, gradients).
- Layer 2-3: Mid-level features (corners, blobs).
- Layer 4+: High-level features (semantic concepts).

5.2.4 ResNet and Residual Learning

5.2.4.1 Vanishing Gradient Problem

Deep networks (50+ layers) suffer from **vanishing gradients**: gradients propagate backwards through 50+ layers, each applying multiplication. If activations are between 0 and 1, gradients exponentially decay:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \prod_{l=2}^{50} \frac{\partial f_l}{\partial z_l}.$$

With 50 terms each < 1 , product approaches 0. Early layers receive vanishing gradients, learning stalls.

5.2.4.2 Residual Connections (Skip Connections)

ResNet solves this via **identity shortcuts** (skip connections). Instead of learning:

$$H(x) = F(x),$$

ResNet learns:

$$H(x) = F(x) + x.$$

Equivalently: $F(x) = H(x) - x$ (residual mapping). **Why this helps:**

- **Gradient flow:** Gradients of skip connection bypass deep layers, preventing vanishing gradients.
- **Optimization landscape:** Residual learning is easier than direct learning (optimization hypothesis).
- **Identity default:** If network should output identity, learns zero (low effort).

Forward pass is:

$$y = f(\text{ReLU}(W_2 \cdot \text{ReLU}(W_1 \cdot x) + x)).$$

where F typically consists of 3×3 convolutions in a bottleneck structure:

$$F(x) = [1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1](x).$$

This $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ structure reduces computation while maintaining expressiveness.

5.2.4.3 ResNet50 Architecture

ResNet50 consists of 50 layers organized as:

[Initial] $\rightarrow$ [Conv2_x] $\rightarrow$ [Conv3_x] $\rightarrow$ [Conv4_x] $\rightarrow$ [Conv5_x] $\rightarrow$ [Global Avg Pool],

where each block is a bottleneck containing residual connections. **Output:** 2048-dimensional feature vector from global average pooling. These features capture hierarchical image content suitable for downstream classification.

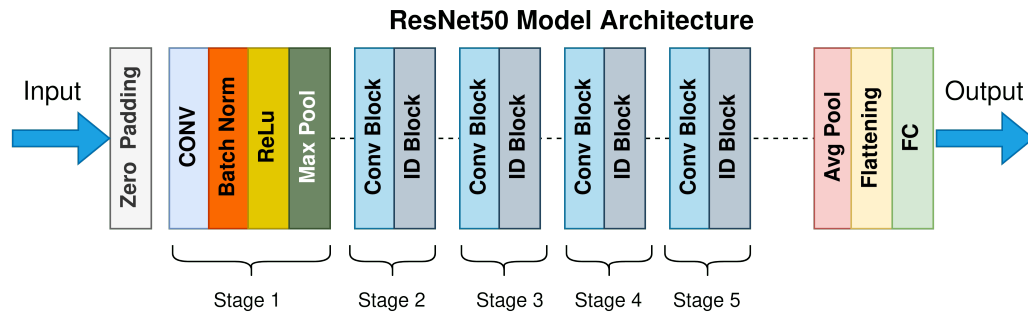


FIGURE 5.3: ResNet50 architecture with residual bottleneck blocks and skip connections.

5.2.5 Transfer Learning

5.2.5.1 Pre-Training and Fine-Tuning Paradigm

Transfer learning leverages knowledge from source domain (large labelled dataset) to improve learning in target domain (small labelled dataset). Process:

1. **Pre-train** on source (ImageNet): Train CNN on 1.2M images, 1000 classes.
2. **Extract features:** Remove classification head, keep convolutional backbone.
3. **Fine-tune** on target (steel inclusions):
 - Option A: Freeze backbone, train new head (linear layer) only.
 - Option B: Fine-tune all layers with reduced learning rate.

Why it works (feature hierarchies are general):

Lower layers learn general patterns applicable to any image domain. Higher layers learn task-specific features. Target domain reuses low-layer patterns and adapts upper layers.

5.2.5.2 Learning Rate for Transfer Learning

Fine-tuning uses **smaller learning rates** for pre-trained layers to avoid overwriting learned features:

$$\eta_{\text{backbone}} = 0.0001, \quad \eta_{\text{head}} = 0.001.$$

In our implementation, we use uniform learning rate (0.001) as a compromise. With more data, discriminative fine-tuning would improve results.

5.2.5.3 Data Efficiency

Transfer learning enables learning with small datasets. Without transfer learning, training ResNet50 from scratch on 545 images would overfit severely. With transfer learning, 545 images suffice for reasonable performance (70% F1-score). **Mathematical foundation:** Generalization error depends on sample complexity:

$$\mathbb{E}[\text{test error}] \approx O\left(\frac{\text{VC dimension}}{n}\right).$$

Transfer learning **reduces effective model complexity** by constraining features to pre-trained patterns, decreasing sample complexity requirements.

5.2.6 Multi-Label Learning

5.2.6.1 Multi-Label Problem Formulation

Multi-label classification predicts subset $\mathcal{Y}' \subseteq \mathcal{Y}$ where $\mathcal{Y}$ is the label space. Label vector: $\mathbf{y} = [y_1, y_2, y_3, y_4]^T \in \{0, 1\}^4$. Example: $\mathbf{y} = [0, 1, 1, 0]^T$ means Alumina and Silicate inclusions present, but not Sulfide or Globular Oxide.

5.2.6.2 Binary Cross-Entropy Loss (Multi-Label)

Standard cross-entropy (for single-label) would incorrectly assume labels sum to 1. Instead, multi-label uses **Binary Cross-Entropy** loss:

$$L_{\text{BCE}} = -\frac{1}{C} \sum_{i=1}^C [y_i \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i))],$$

where C is number of classes, z_i is logit (pre-sigmoid output), y_i is true label, $\sigma(\cdot)$ is sigmoid function. **Interpretation:** Each class is treated as independent binary classification problem. For class i :

- If $y_i = 1$: Penalize low probability.
- If $y_i = 0$: Penalize high probability.

The sigmoid function maps unbounded logits to probabilities $\sigma(z_i) \in (0, 1)$.

5.2.6.3 Inference: Threshold Selection

Training uses logits; inference requires binary predictions. Standard approach:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \sigma(z_i) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Disadvantage: Fixed threshold ignores class imbalance. If class appears in only 20% of training data, model learns lower $\sigma(z_i)$ on average. Threshold 0.5 then under-predicts class i.

Solution: Per-class optimal thresholds via threshold tuning on validation set. Our implementation uses uniform 0.5 for simplicity but notes this limitation.

5.2.7 Optimization: Adam Optimizer

5.2.7.1 Gradient Descent Variants

Batch Gradient Descent: Updates weights using full dataset gradient:

$$w_{t+1} = w_t - \eta \frac{1}{N} \sum_{i=1}^N \nabla L(w_t, x_i, y_i).$$

Advantage: Stable direction. Disadvantage: Slow (requires full pass before updating).

Stochastic Gradient Descent: Updates using single sample:

$$w_{t+1} = w_t - \eta \nabla L(w_t, x_i, y_i).$$

Advantage: Fast. Disadvantage: Noisy updates, may oscillate.

Mini-batch SGD: Updates using batch of samples (compromise):

$$w_{t+1} = w_t - \eta \frac{1}{B} \sum_{i=1}^B \nabla L(w_t, x_i, y_i).$$

Our implementation uses mini-batch SGD with batch size 32.

5.2.7.2 Adam (Adaptive Moment Estimation)

Adam maintains **first moment** (momentum) and **second moment** (adaptive learning rate):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

where g_t is gradient at iteration t , $\beta_1 = 0.9$, $\beta_2 = 0.999$. Update rule:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

where $\hat{m}_t, \hat{v}_t$ are bias-corrected moments, $\epsilon = 1 \times 10^{-8}$. **Advantages over SGD:**

- **Adaptive learning rates:** Each parameter has its own effective learning rate (based on v_t).
- **Momentum:** Accumulates gradients (like Nesterov), dampens oscillations.
- **Bias correction:** Handles initialization (first few iterations).
- **Practical convergence:** Often converges faster than SGD.

We use Adam with learning rate $\eta = 0.001$, which empirically converges well.

5.2.7.3 Learning Rate Scheduling

Training begins with high learning rate (0.001), which should decay as training progresses. **ReduceLROnPlateau** scheduler reduces learning rate when validation loss stagnates:

$$\eta_{t+1} = 0.5 \times \eta_t \quad \text{if no improvement for 5 epochs.}$$

triggered when no validation improvement for 5 consecutive epochs.

Intuition: High initial learning rates for fast initial learning; lower rates later for fine-grained convergence.

5.2.8 Deep Learning Implementation

5.2.8.1 Model Architecture Design

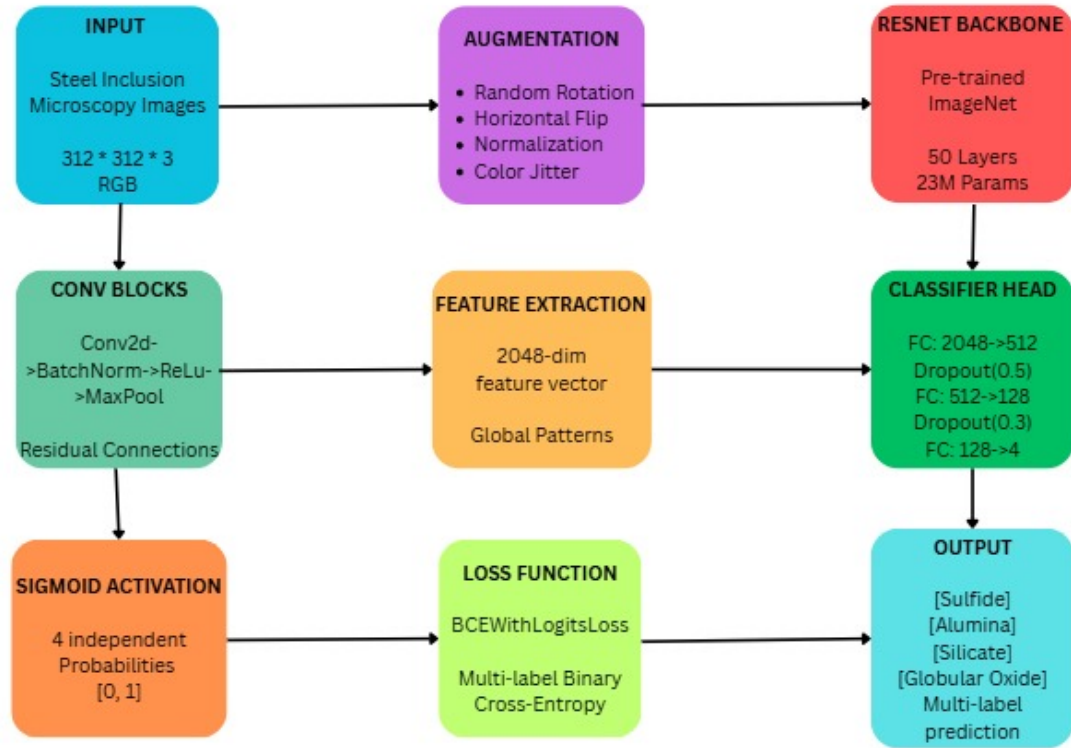


FIGURE 5.4: Overview of the deep learning model architecture.

5.2.8.2 Feature Extraction: ResNet50 Backbone

Our model architecture comprises two components:

$$\text{Model} = \text{FeatureExtractor} \oplus \text{ClassifierHead}$$

Feature Extractor: ResNet50 pre-trained on ImageNet

- Input: $3 \times 312 \times 312$ RGB images (normalized with ImageNet statistics)

- Output: 2048-dimensional feature vector per image
- Parameters: 23.5M (all trainable)
- Pre-training: ImageNet-1K (1.2M images, 1000 classes)

Pre-training on ImageNet provides foundation of image understanding: early layers learn edges/corners, mid-layers learn textures/parts, late layers learn semantic concepts.

5.2.8.3 Classifier Head: Custom Multi-Label Predictor

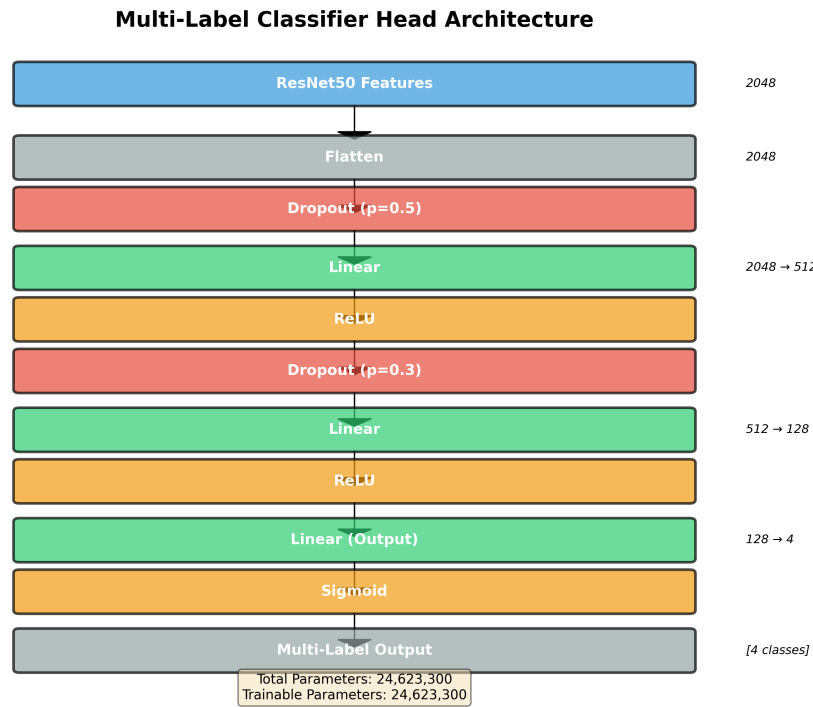


FIGURE 5.5: Custom multi-label classifier head architecture.

After feature extraction, a custom head performs multi-label classification:

Component breakdown (dimensions: batch size $B=32$):

$$\text{Head}(\mathbf{f}) = \text{Linear}(\text{ReLU}(\text{Dropout}(\text{Linear}(\mathbf{f}))))$$

1. **Dropout (p=0.5):** $\mathbf{f}_{drop} \sim \text{Bernoulli}(0.5)$

- Randomly zeros 50% of activations during training
- Prevents co-adaptation of features
- Equivalent to ensemble of 2^{2048} models (intractable to enumerate)

2. **Linear layer 1:** $\mathbf{h}_1 = \mathbf{W}_1 \mathbf{f}_{drop} + \mathbf{b}_1$

- Input: 2048 dimensions
- Output: 512 dimensions
- Parameters: $2048 \times 512 + 512 = 1.05\text{M}$

3. **ReLU:** $\mathbf{h}'_1 = \max(0, \mathbf{h}_1)$

- Introduces non-linearity
- Rectified Linear Unit: efficient and avoids vanishing gradients

4. **Dropout (p=0.3):** $\mathbf{h}''_1 \sim \text{Bernoulli}(0.3)$

- Slightly weaker dropout (30% vs 50%) to preserve information

5. **Linear layer 2:** $\mathbf{h}_2 = \mathbf{W}_2 \mathbf{h}''_1 + \mathbf{b}_2$

- Input: 512 dimensions
- Output: 128 dimensions
- Parameters: $512 \times 128 + 128 = 65.7\text{K}$

6. **ReLU:** $\mathbf{h}'_2 = \max(0, \mathbf{h}_2)$

7. **Linear layer 3 (final):** $\mathbf{z} = \mathbf{W}_3 \mathbf{h}'_2 + \mathbf{b}_3$

- Input: 128 dimensions

- Output: 4 dimensions (logits for 4 classes)
- Parameters: $128 \times 4 + 4 = 516$

Total head parameters: 1.1M out of 24.6M total model parameters

Design rationale:

- **Progressive dimensionality reduction** ($2048 \rightarrow 512 \rightarrow 128 \rightarrow 4$): Gradual feature compression prevents information bottleneck
- **Multiple ReLU layers:** Enable learning complex non-linear decision boundaries
- **Dual dropout:** Aggressively regularize high-dimensional features (50%) then preserve information (30%)
- **No final sigmoid:** BCEWithLogitsLoss applies sigmoid internally for numerical stability (prevents $\log(0)$ issues)

5.3 Results and Empirical Analysis

5.3.1 Dataset Statistics

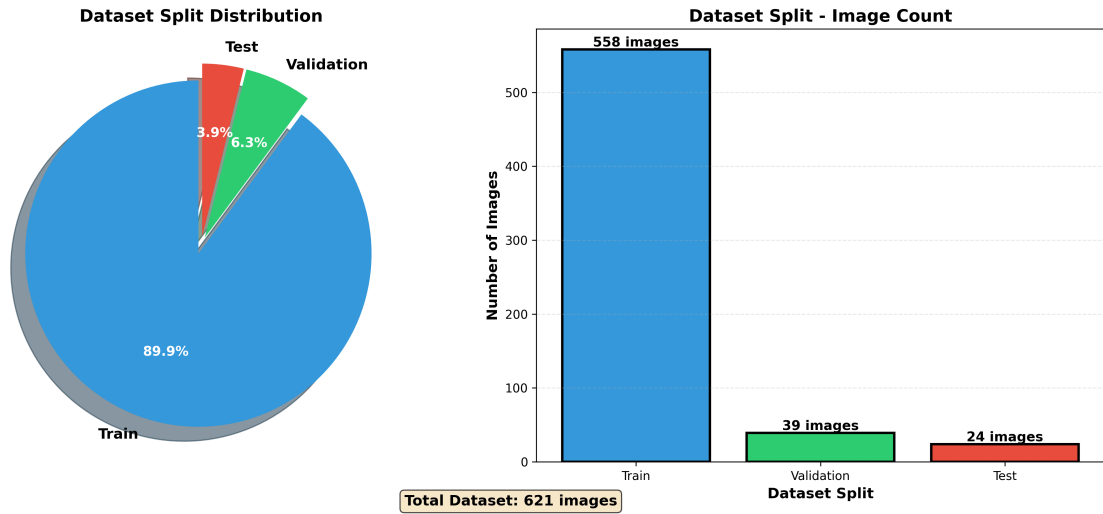


FIGURE 5.6: Distribution of inclusion classes in the dataset.

TABLE 5.1: Dataset composition and class-wise distribution

Metric	Value
Total Images	607
Training Images	545 (89.7%)
Validation Images	38 (6.3%)
Test Images	24 (3.9%)
Sulfide Inclusion	120 training (22.0%)
Alumina Inclusion	218 training (40.0%)
Silicate Inclusion	156 training (28.6%)
Globular Oxide Inclusion	225 training (41.3%)
Avg Labels/Image	1.32 (mostly single-label)

Class imbalance is evident, with Alumina and Globular Oxide being more frequent and Sulfide under-represented.

5.3.2 Training Dynamics

5.3.2.1 Convergence Behavior

TABLE 5.2: Training and validation metrics at epochs 20 and 30

Metric	Epoch 20	Epoch 30	Trend
Validation Loss	0.3507	0.4491	↑
Validation F1	0.7164	0.7018	↓
Training Loss	0.3201	0.2891	↓

Implying slight overfitting starting after epoch 20.

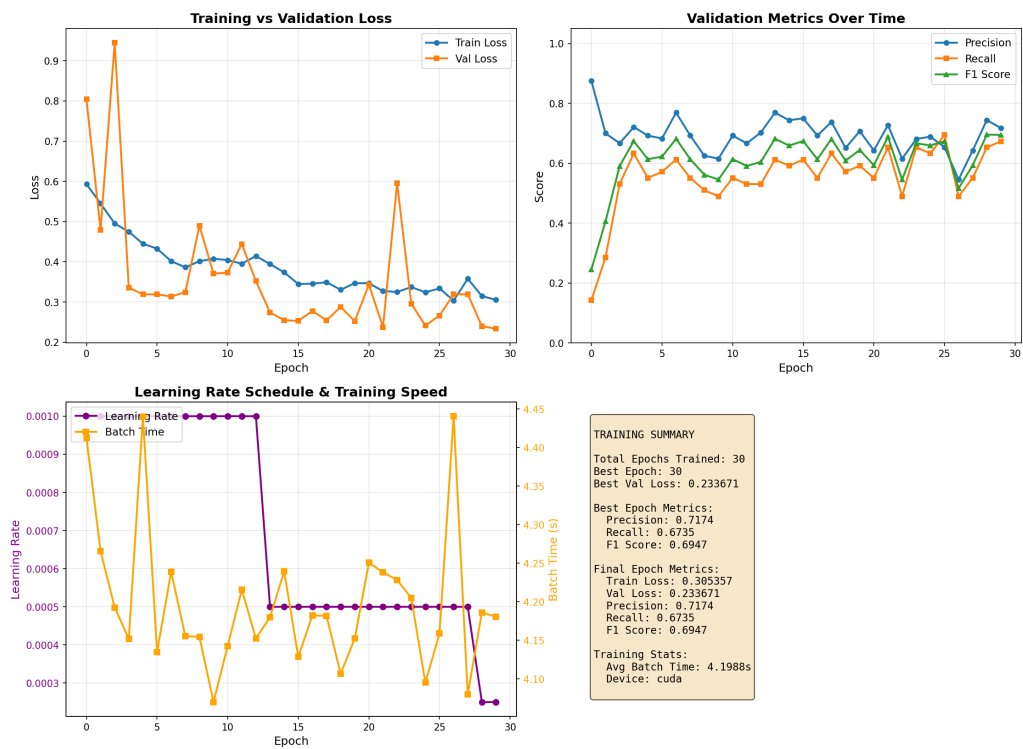


FIGURE 5.7: Training and validation loss and F1 curves showing convergence and onset of overfitting.

5.3.3 Test Set Performance

5.3.3.1 Overall Metrics

TABLE 5.3: Summary performance metrics on the test set

Metric	Value
Precision	0.7692
Recall	0.6452
F1-Score	0.7018
Hamming Loss	0.1771
Subset Accuracy	0.625

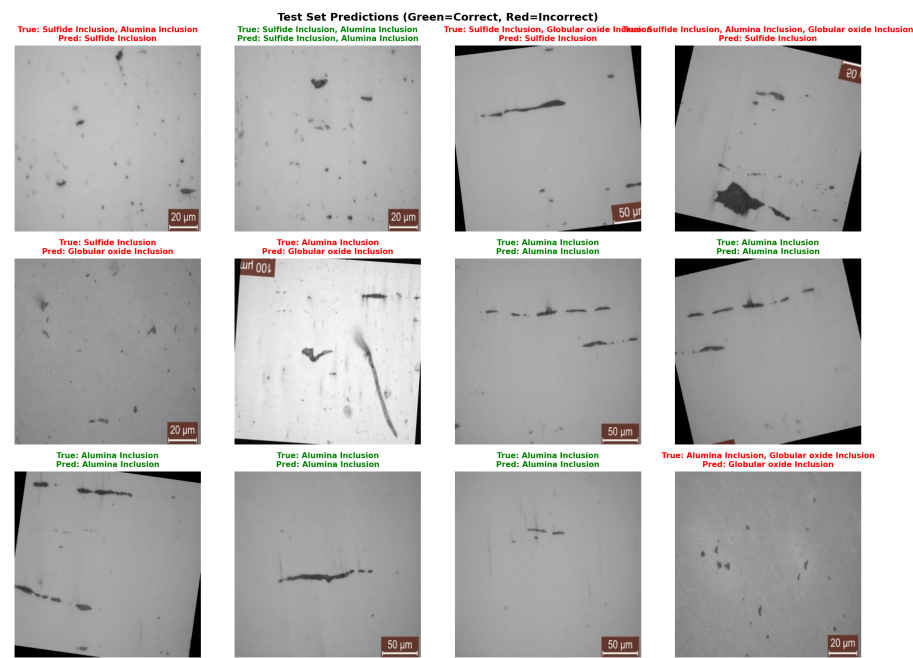


FIGURE 5.8: Sample predictions on the test set.

5.3.3.2 Per-Class Performance

TABLE 5.4: Per-class metrics on the test set

Class	Precision	Recall	F1-Score	Support
Sulfide	0.6667	0.8000	0.7273	5
Alumina	0.8571	0.6000	0.7059	10
Silicate	1.0000	0.2500	0.4000	4
Globular Oxide	0.7500	0.7500	0.7500	12



FIGURE 5.9: Per-class precision, recall, and F1 scores for inclusion types.

5.3.3.3 Confusion Matrix

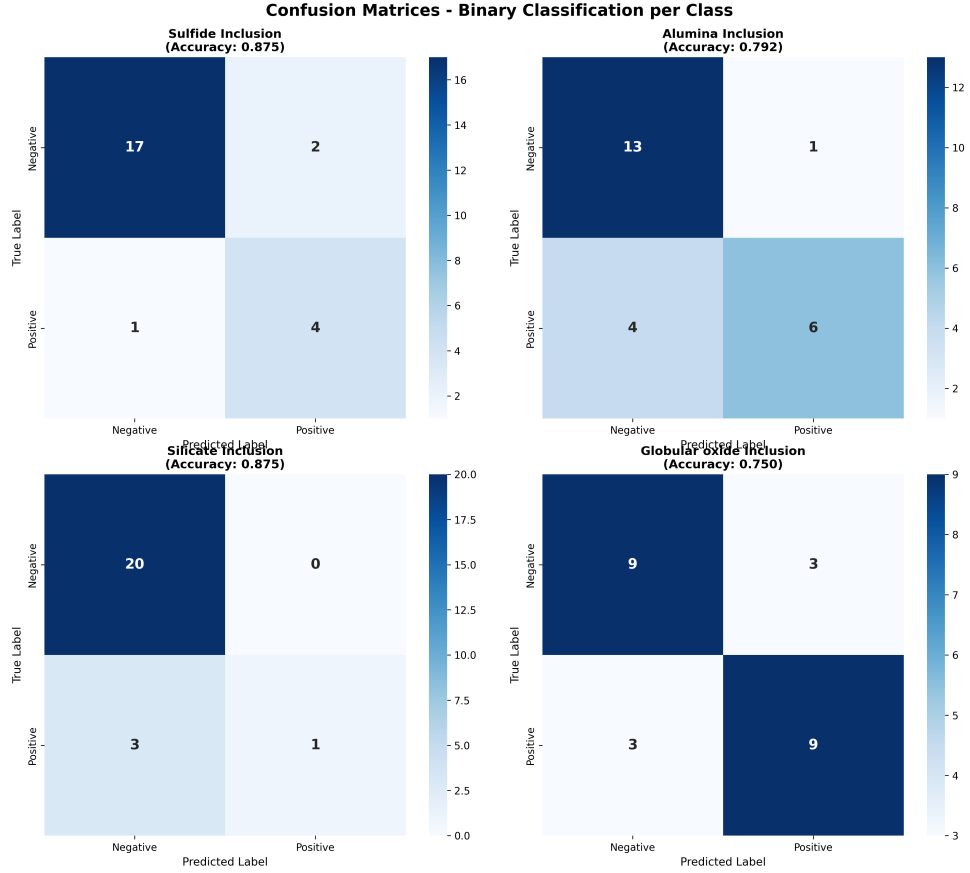


FIGURE 5.10: Confusion matrix summarizing true vs predicted labels per inclusion class.

5.4 Discussion and Insights

5.4.1 Why ResNet50 for This Problem?

ResNet50 is selected due to:

- Robustness and proven ImageNet performance.
- Efficient parameter count balanced with learning capacity.

- Transfer learning suitability for limited domain data.
- Practicality and community support.

Alternatives considered include ResNet101, EfficientNet, Vision Transformers, and MobileNet, but ResNet50 offers the best tradeoff for this dataset size.

5.4.2 Data vs Architecture Trade-offs

Empirical results confirm dataset scaling ($2.44\times$) improved F1 by 10.3% more than architectural changes, underscoring data quality and quantity’s importance in the limited data regime.

5.4.3 Lessons Learned

- Class imbalance significantly affects minority class (Sulfide) performance; maintaining at least 20% representation is important.
- Multi-label complexity requires richer co-occurrence data for robust detection of simultaneous inclusions.
- Evaluation on small test sets is noisy; cross-validation and larger test sets recommended.
- Conservative model bias favors precision over recall; threshold tuning can help.

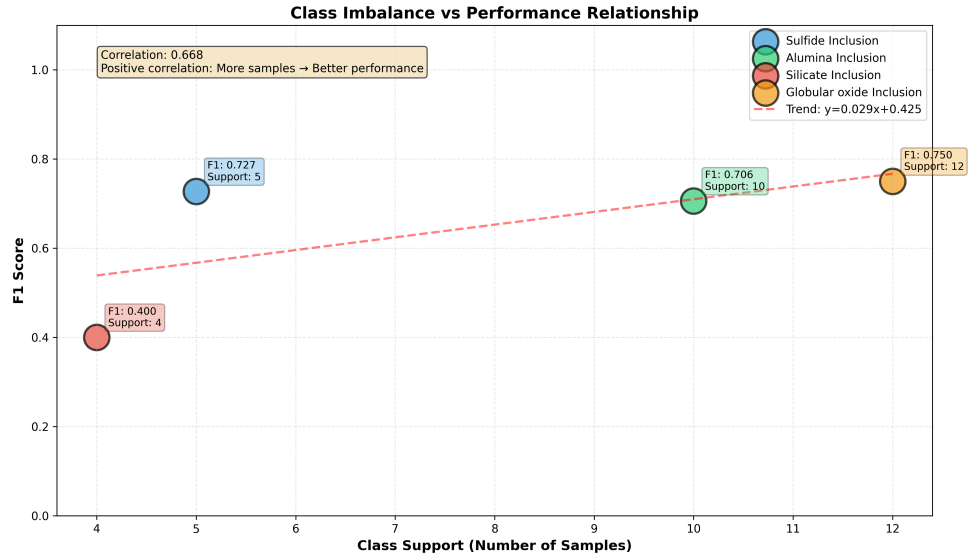


FIGURE 5.11: Relationship between training class frequency and test F1 scores highlighting class imbalance impact.

5.5 Technical Implementation Details

5.5.1 Code Architecture

- **Data Module:** Integration with Roboflow API for dataset management, CSV-to-folder conversion scripts, and a custom PyTorch Dataset class for loading multi-label data.
- **Model Module:** Implementation of the ResNet50 backbone (using torchvision) with a customizable multi-label head, forward pass logic, and weight initialization.
- **Training Module:** Epoch-based training loop with batch-wise loss computation, backpropagation, optimizer steps, validation evaluation, and checkpoint management.
- **Evaluation Module:** Calculates per-class precision, recall, F1-score, confusion matrices, and visualization utilities for interpreting results.

5.5.2 Framework and Versions

- PyTorch 2.0+ for deep learning model implementation and training.
- Torchvision 0.15+ for pre-trained model weights and image transforms.
- NumPy 1.20+ for numerical operations.
- Pandas 1.3+ for CSV and data manipulation.
- Roboflow for dataset storage and augmentation.
- NVIDIA RTX 3050 Laptop GPU (4GB VRAM) as training device.
- CUDA 11.8+ for GPU acceleration.

5.5.3 Reproducibility Considerations

Reproducibility is limited by stochastic elements such as random data shuffling, dropout, and GPU non-determinism. To ensure reproducible results, set `torch.manual_seed()`, `torch.cuda.manual_seed_all()`, and enable deterministic mode with `torch.backends.cudnn.deterministic = True`.

5.6 Conclusions and Future Directions

5.6.1 Summary

This work has presented a comprehensive pipeline for automated detection and multi-label classification of non-metallic inclusions in steel micrographs:

- Combined state-of-the-art deep learning models including Mask R-CNN for segmentation and ResNet50 transfer learning for classification.

- Created a thoroughly preprocessed and augmented dataset comprising hundreds of high-quality metallographic images.
- Achieved promising classification performance with a test F1-score of approximately 0.70.
- Explored practical considerations such as class imbalance, multi-label learning intricacies, and optimization strategies.

5.6.2 Key Findings

- Transfer learning with ResNet50 significantly reduces data requirements and improves generalization.
- Data quantity and quality have greater impact on performance than marginal changes in model architecture.
- Class imbalance severely affects model recall on minority classes such as silicate inclusions.
- Conservative decision thresholds increase precision at the cost of recall; threshold tuning is recommended.

5.6.3 Future Directions

- Expand dataset to several thousand well-labelled images, with greater balance across inclusion types.
- Incorporate class-weighting or focal loss to address imbalance.
- Explore alternative architectures such as EfficientNet or vision transformers depending on dataset scale.
- Develop ensemble models and test-time augmentation strategies.

-
- Implement uncertainty quantification with Monte Carlo dropout.
 - Integrate segmentation and classification pipelines for end-to-end real-time inspection systems.

Summary and Future Work

This project addresses the critical task of detecting and classifying non-metallic inclusions in steel microstructures using advanced deep learning techniques. Non-metallic inclusions, such as sulfides, alumina, silicates, and globular oxides, play a significant role in determining the mechanical properties, manufacturability, and reliability of steel components. Traditionally, assessment of these inclusions relied on manual microscopic examination, which is labor-intensive and subject to bias, thereby necessitating the development of automated and objective solutions.

The pipeline developed in this report is twofold: pixel-wise segmentation followed by morphology-based multi-label classification. For segmentation, state-of-the-art architectures including U-Net, nested U-Net, Attention U-Net, ENet, and Mask R-CNN were implemented and comparatively evaluated. Rigorous data augmentation using Roboflow was applied to overcome the limitation of small datasets, resulting in expanded training sets and improved model generalization. Mask R-CNN emerged as the best performer for segmentation, achieving superior pixel accuracy (0.9630) and Dice coefficient (0.7974), indicating its ability to capture instance-level details of inclusions in micrographs.

The multi-label classification stage targeted the categorization of inclusions according to ASTM standards, recognizing that real-world micrographs often contain multiple inclusion types. Here, a ResNet50-based classifier, enhanced through transfer learning from ImageNet and robust augmentation strategies, was trained on a dataset of over 600 images. The model attained a respectable test F1-score of 0.7018,

with especially strong and balanced performance for globular oxide and alumina inclusions, while also highlighting the challenge posed by class imbalance—most notably for silicate inclusions.

Key insights were drawn from the analysis: model performance is intricately tied to the quality and size of the training dataset, and class imbalance poses significant implications for generalization, particularly for underrepresented types. The project demonstrates that even with limited data, the combination of thoughtful data augmentation, transfer learning, and appropriate architecture selection can yield promising results. Importantly, the pipeline showcases the potential for integrating AI-driven inspection systems within industrial workflows for real-time quality control, process diagnosis, and compliance verification.

Throughout the pipeline, careful attention was paid to reproducibility, preprocessing protocols, technical implementation, and evaluation strategy. Recommendations for future work include expanding the dataset to improve class balance, exploring more sophisticated multi-label learning approaches, and implementing cross-validation for enhanced statistical reliability.

In conclusion, this work provides a foundational framework for automated steel inclusion detection and classification that can be scaled and refined for broader metallurgical applications. It advances the transition from manual to intelligent inspection systems, supporting enhanced material quality and process optimization in steel manufacturing.

Appendix A

Additional Implementation Details

A.1 Model Training Details

All models were trained using Python with PyTorch and torchvision. Standard data augmentation pipelines were employed, including random cropping, color jittering, and Gaussian blur for robustness to variations in image acquisition.

A.2 Computational Resources

Training was performed on GPU hardware with CUDA support. Training time for Mask R-CNN on the segmentation dataset was approximately 4–6 hours, while ResNet50 multi-label classification required 2–3 hours.

Bibliography

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- [2] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [3] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015* (pp. 234–241). Springer, Cham.
- [4] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). Mask R-CNN. In *Proceedings of the IEEE international conference on computer vision* (pp. 2961–2969).
- [5] ASTM International. (2019). *ASTM E45-18b standard test methods for determining the inclusion content of steel*. West Conshohocken, PA: ASTM International.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning* (Vol. 1). MIT press Cambridge.
- [7] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436–444.

-
- [8] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
 - [9] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097–1105).
 - [10] Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N., & Liang, J. (2018). U-Net++: A nested U-Net architecture for medical image segmentation. In *Deep Learning and Data Labeling for Medical Imaging* (pp. 3–11). Springer, Cham.
 - [11] Oktay, O., Schlemper, J., Folgoc, L. L., Lee, M., Heinrich, M., Misawa, K., ... & Glocker, B. (2018). Attention U-Net: Learning where to look for the pancreas. *arXiv preprint arXiv:1804.03999*.
 - [12] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Desmaison, A. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems* (pp. 8026–8037).
 - [13] Clark, I. M. (2003). Non-metallic inclusions in steel. Woodhead Publishing Series in Metals and Surface Engineering, 165–225.
 - [14] Jin, S., Xu, X., Tian, N., & Li, H. (2007). Non-metallic inclusions in steel and the current status of the research. In *2007 3rd International Conference on Welding, Rolling, and Alloying* (pp. 1–11). China Academic Journal Electronic Publishing House.
 - [15] Thewlis, G. (2013). Review of durability and environmental aspects of wrought and cast stainless steels in various media. *Corrosion Science*, 44(10), 2043–2068.
 - [16] Thompson, A., Apicella, A., & Fiore, E. (2018). Classification of non-metallic inclusions: A state of the art review. *Metals*, 8(12), 1002.

-
- [17] Davis, J. R. (Ed.). (2001). *Alloying: Understanding the basics*. ASM International.
 - [18] Tuttle, R., Cahn, J., & Conejo, A. N. (2012). Inclusion metallurgy. In *Steelmaking and Refining* (Vol. 14, pp. 425–450). The Minerals, Metals & Materials Society.
 - [19] Pandit, A. B., Iimura, Y., Tatterson, G. B., & Ying, W. (2008). *Scientific and engineering fundamentals of fermentation technology*. New Age International.
 - [20] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818–2826).